

光伏微网中计及信息到达与非对称结算的储能—购电滚动优化

摘 要

本文研究含光伏、储能和外网购电的居民微网运行问题。题目的核心不是简单确定“光伏、储能、外网”的固定供电顺序，而是在每个十分钟时段满足能量守恒，并利用储能把低价或富余光伏电量转移到更有价值的时段。由此，本文先从母线能量平衡和储能状态转移建立统一骨架，再依据四问中信息到达时刻与结算规则的变化逐层扩展模型。

对问题一，在价格、负荷和光伏均确定的条件下建立含充放电互斥状态的混合整数线性规划。模型所得日购电量为 **59 482.70 kWh**，日购电费用为 **35 126.95 元**，较无储能基线降低 **26.90%**；能量平衡和状态转移最大残差分别为 2.27×10^{-13} kWh 和 8.31×10^{-13} kWh。连续松弛与主模型在本实例上的目标值相同，但主模型仍保留互斥约束以排除其他数据条件下的非物理解。

对问题二，附件只给出实际负荷与实际光伏，不能在每日 0:00 预先读取。因此将每日决策拆为严格两阶段：第一阶段固定整日购电及充放电计划，第二阶段在实际曲线实现后只允许以 5 倍电价紧急购电，并将富余电量记为弃电。利用决策日前历史构造联合净负荷场景，以整日残差块保留时序相关，并用 48 小时延伸窗传递日末储能价值。正式期总费用为 **15 258 022.89 元**，其中紧急购电 **404 258.73 kWh**。

对问题三，0:00、6:00、12:00、18:00 的新光伏预报依次改变未执行时段的信息集，因而采用确定性 MILP-MPC。每次调整只作用于未来时段，增购按 $1.5p_t$ 、减购按上一版计划退回 $0.5p_t$ ，最终缺口仍按 $5p_t$ 结算。全年消融结果表明 **0:00、6:00、12:00 与 18:00** 的总费用最低，说明全部日内预报均具有覆盖其调整成本的净价值。场景 MPC 与 CVaR-MPC 未在缺少稳定样本外增益的情况下进入主模型。

对问题四，实时价格的未来实现值在决策时不可见。本文以仅用历史价格的因果预测驱动 Q4-2 和 Q4-3，分别得到总费用 **16 124 373.46 元**、**16 550 967.30 元**；同时计算只放宽价格信息的反事实基准和全外生信息 oracle 严格下界，量化可获得信息的价值。全部结果均通过跨日 SOC、充放电互斥、费用账本和未来信息泄漏检查，并按附件 5 回填五份结果表。

关键词：光伏微网；储能调度；两阶段随机规划；模型预测控制；因果预测；混合整数线性规划

1 问题重述

小区微网由光伏、储能、外网和负荷组成。光伏可以直接向负荷供电，也可以给储能充电；储能放电和外网购电共同弥补剩余缺口。题目给出 10 分钟粒度的电价、负荷、光伏数据，并规定储能额定容量为 12 000 kWh，允许运行区间为 1 200—10 800 kWh，最大充放电功率为 5 000 kW，充、放电效率均为 90%。

问题一要求在典型日曲线完全已知时确定计划购电量与储能策略，且 0:00 与 24:00 的储电量相同。问题二把负荷和光伏改为逐日变化的实际曲线，每日 0:00 制定全天计划，供电不足时按 5 倍电价紧急购电。问题三在每日 0:00、6:00、12:00、18:00 更新未来 24 小时光伏预测，并允许按非对称价格调整未执行的购电计划。问题四进一步把固定分时电价换为逐日实时波动价格，要求分别重算问题二和问题三。

2 问题分析与模型路线

2.1 模型为何从题目中自然产生

每个时段的电量有明确去向：外网购电、光伏发电和储能放电进入母线，负荷、储能充电和弃电离开母线；相邻时段只通过储电量连接。容量、功率和购电量均有线性边界，费用是购电量与单价的线性乘积，问题三的增减购电费用可以用正负差额拆成分段线性形式。因此，题目首先给出的是一个线性时序优化骨架；只有“同一套储能不能同时充放电”需要二元状态。采用 MILP 是物理结构的结果，而不是套用算法模板。储能 SOC 对微网经济调度的约束作用也与相关研究结论一致 [1]。

2.2 四问之间的递进关系

问题	新增题干事实	数学含义	相应模型部件
Q1	日曲线完全给定、首尾 SOC 相同	单日确定性闭环	确定性 MILP
Q2	0:00 决策、实际偏差 5 倍结算	决策与实现分离	严格两阶段随机 MILP
Q3	每 6 小时发布新光伏预测	信息集滚动、已执行决策冻结	确定性 MILP-MPC
Q4	实时价格未来未知	价格预测必须因果	因果价格策略与信息下界

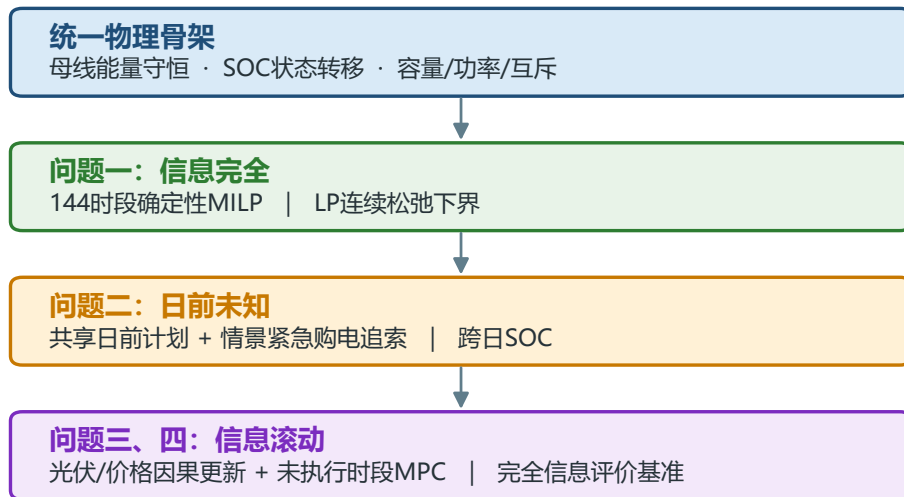


图 2-1 四问模型由题目信息结构逐层生长

2.3 不采用若干复杂方法的理由

遗传算法、粒子群等启发式方法不能利用本题的线性结构，难以给出可验证的全局下界；动态规划需要离散 SOC，存在离散误差和维数增长；强化学习需要大量交互样本，而本题只有一年数据且物理约束和费用函数均已显式给出。鲁棒优化和 CVaR 可以作为风险增强，但其不确定集半径、置信水平和风险权重若不能由滚动样本外结果稳定识别，就会引入比题目本身更强的主观假设。因此本文将复杂模块设置为“以证据换复杂度”的待证增强，而不默认叠加。

3 模型假设与数据处理

3.1 基本假设

1. 每个 10 分钟时段内的功率取附件所给平均值，时段能量等于功率乘 1/6 小时。
2. 光伏优先直接参与母线平衡；模型不预设固定的供电顺序，而由费用与约束共同决定光伏、储能和外网的组合。
3. 题目未给出向外网售电及收益，故富余电量只能充电或弃置，弃电收益为 0。
4. 充电量与放电量定义在交流母线侧，充、放电效率分别为题定的 0.9。
5. Q2—Q4 的 SOC 跨日连续，只有 Q1 使用题目明确给出的日初日末相等条件。

6. 所有预测器、标准化参数、场景残差和模型选择只使用决策时刻以前的数据；当天未来实际值只用于事后结算和评价。

3.2 时间与单位对齐

记每日时段集合为

$$\mathcal{T} = \{1, 2, \dots, 144\}, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

原始附件按“00:10, 00:20, ..., 0:00+1”给出 144 项，结果模板按 144 个时段留出位置。题面未明确功率是端点瞬时值还是区间平均值，且模板末时段标签存在书写差异。为避免人为移动数据，本文采用可复核的同序号对齐：附件第 t 项与模板第 t 个时段对应，回填时保留模板顺序，不做循环平移。

负荷与光伏功率分别记为 $L_{d,t}$ 、 $P_{d,t}^{\text{pv}}$ ，对应的十分钟电量为

$$\ell_{d,t} = L_{d,t} \Delta t, \quad s_{d,t} = P_{d,t}^{\text{pv}} \Delta t. \quad (2)$$

4 符号说明

符号	含义	单位
$G_{d,t}$	0:00 计划或当前有效的外网购电量	kWh
$C_{d,t}$	母线送入储能的充电量	kWh
$D_{d,t}$	储能向母线输出的放电量	kWh
$R_{d,t}$	实际供电不足时的紧急购电量	kWh
$W_{d,t}$	不能利用的富余电量	kWh
$E_{d,t}$	时段 t 开始前的储电量	kWh
$z_{d,t}$	充放电互斥状态，1 表示允许充电	—
$p_{d,t}$	对应时段外网购电单价	元/kWh
ω	净负荷情景编号	—
$q_{d,\omega}$	日期 d 的情景概率	—

5 统一物理模型

5.1 母线能量守恒

实际实现时，每一时段满足

$$G_{d,t} + R_{d,t} + s_{d,t} + D_{d,t} = \ell_{d,t} + C_{d,t} + W_{d,t}. \quad (1)$$

式 (1) 明确了“弃光”的含义：当光伏与既定供电量超过负荷和可执行充电量时，多出的能量记为 $W_{d,t}$ 。若把光伏实际使用量单独设为 $P^{\text{pv,use}}$ ，则有 $P^{\text{pv}} = P^{\text{pv,use}} + W$ ，两种平衡写法完全等价。

5.2 储能状态转移与互斥

由于 $C_{d,t}$ 、 $D_{d,t}$ 位于母线侧，题定双效率 90% 给出

$$E_{d,t+1} = E_{d,t} + 0.9C_{d,t} - \frac{D_{d,t}}{0.9}. \quad (2)$$

容量与功率边界为

$$1200 \leq E_{d,t} \leq 10800, \quad (3)$$

$$0 \leq C_{d,t} \leq \frac{5000}{6}z_{d,t}, \quad 0 \leq D_{d,t} \leq \frac{5000}{6}(1 - z_{d,t}), \quad z_{d,t} \in \{0, 1\}. \quad (4)$$

互斥约束不是为了形式复杂，而是阻止模型用“同时充放电产生损耗”的方式虚假消纳免费光伏。若去掉 $z_{d,t}$ ，连续松弛只能作为下界与消融对照。

5.3 跨日 SOC

题目只给出一次真实初值

$$E_{2025-01-01,1} = 6000, \quad (3)$$

Q2—Q4 按自然时间顺序运行，并满足

$$E_{d+1,1} = E_{d,145}. \quad (5)$$

1 月用于从 6000 kWh 热启动与校准，正式汇总期为 2 月 1 日至 12 月 31 日，共 334 日。为了避免有限日内窗口在 24:00 前无代价放空储能，0:00 采用 48 小时延伸窗估计日末续接状态，只执行前 24 小时；这不等于强迫每天首尾相等。

6 问题一：确定性日内调度

6.1 目标函数与边界条件

Q1 的价格、负荷和光伏均已给定，且不需要紧急购电，故目标为

$$\min C_1 = \sum_{t=1}^{144} p_t G_t, \quad (6)$$

并满足式 (1) — (4)，其中 $R_t = 0$ ，以及

$$E_1 = E_{145} = 6000. \quad (7)$$

6.2 求解结果与解释

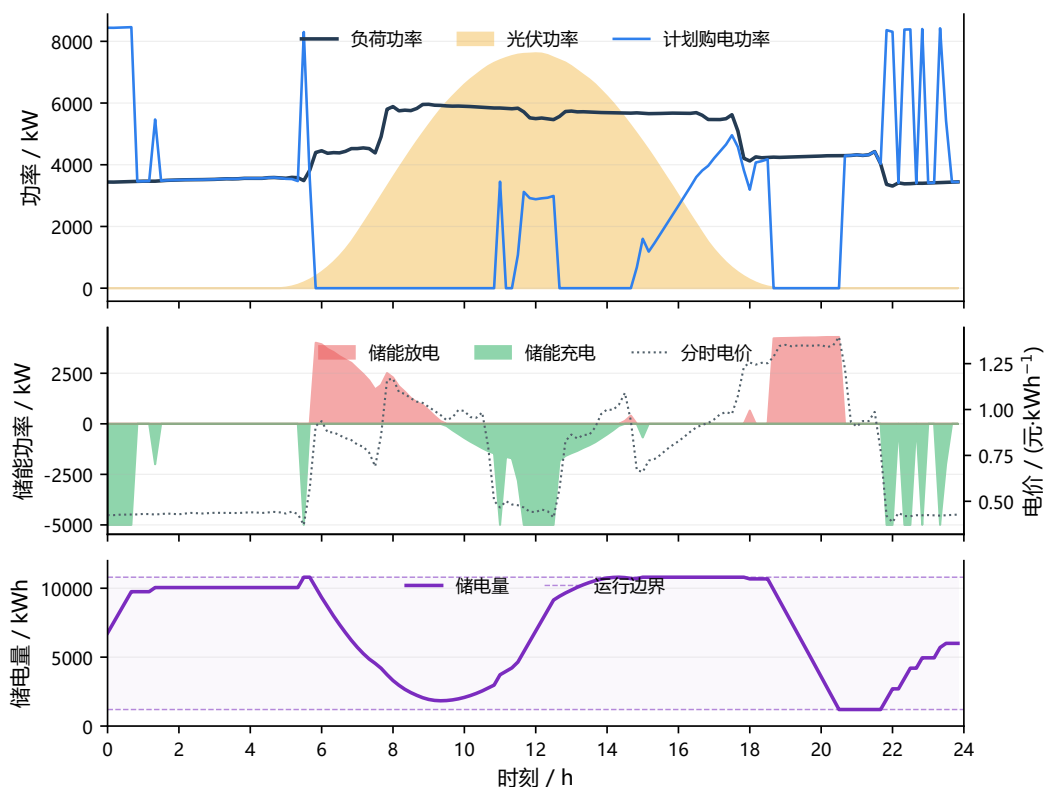


图 6-1 典型日功率、储能动作与 SOC 轨迹

表 6-1 指定时段购电量及全天汇总

表 6-2 储能分段充/放电量及首末储电量

模型在低价时段或光伏富余时充电，在高价且负荷缺口较大时放电。某些时段计划购电量出现陡变，是分时电价台阶和 5 000 kW 功率上限共同作用的结果，不表示供电中断。LP 在本组数据上自然落在互斥解上，因而与 MILP 同值；但这只是数据特例，不

指标	MILP 主模型	LP 连续松弛	无储能基线
日购电费用/元	35 126.95	35 126.95	48 052.05
日购电量/kWh	59 482.70	59 482.70	—
充电量/kWh	20 740.67	20 740.67	0
放电量/kWh	16 799.94	16 799.94	0
同时充放电时段/个	0	0	0

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00-10:10	0.00	12:00-12:10	486.40	14:00-14:10	0.00
16:00-16:10	394.93	18:00-18:10	636.98	20:00-20:10	0.00
全天购电量	59 482.70	全天购电费	35 126.95		

时段	充/放电量 (kWh)	时段	充/放电量 (kWh)
0:00-4:00	4500.00/0.00	4:00-8:00	833.33/6365.84
8:00-12:00	4787.96/1703.00	12:00-16:00	5286.04/91.10
16:00-20:00	0.00/5780.13	20:00-24:00	5333.33/2859.87
0:00 储电量	6 000.00	24:00 储电量	6 000.00

能把互斥约束从正式模型中删除。

7 问题二：严格日前两阶段随机调度

7.1 信息集与两阶段边界

附件 2 只有当天结束后才能完整观测的实际曲线。日期 d 的 0:00 信息集定义为

$$\mathcal{I}_{d,0}^{(2)} = \{E_{d,1}, p_{1:144}, \text{日历特征}, (L_{j,:}, P_{j,:}^{\text{PV}})_{j < d}\}. \quad (8)$$

第一阶段变量 $G_{d,t}, C_{d,t}, D_{d,t}, E_{d,t}, z_{d,t}$ 对所有情景共享；第二阶段只有紧急购电 $R_{d,t}^\omega$ 和弃电 $W_{d,t}^\omega$ 可以随情景变化。两阶段结构在含可再生能源不确定性的微网调度研究中具有明确的“先决策一后追索”含义 [7-9]，但本文进一步按题意把追索权限收窄为紧急购电与弃电：

$$G_{d,t} + R_{d,t}^\omega + D_{d,t} - C_{d,t} - W_{d,t}^\omega = n_{d,t}^\omega, \quad (9)$$

其中 $n_{d,t}^\omega = \ell_{d,t}^\omega - s_{d,t}^\omega$ 。不设置情景相关的储能补救，是因为题目只明确授权供电不足时紧急购电，也使 Q2 与 Q3 的“日内可调整”形成清楚边界。

7.2 因果预测与联合场景

净负荷具有显著日周期和周周期。预测误差会通过储能调度放大为经济损失，因此必须在后续费用结果中检验预测是否真正有用 [3]。本文比较 1 日—7 日季节加权基线与带日历、滞后项的岭回归；每月首日只使用此前最近 14 个完整日滚动验证，以 WAPE 为主指标，并在最优值 1% 范围内优先采用更简单的季节基线。对已完成日期保存整日残差向量 $\mathbf{e}_j$ ，构造

$$n_{d,t}^\omega = \hat{n}_{d,t} + e_{j\omega,t}. \quad (10)$$

整日抽样保留 144 个时段误差的共同升降和峰谷偏移，避免逐点独立抽样破坏时序相关。情景概率采用经验频率，情景数由 10、20、30 个候选的分布统计稳定性确定最小规模。

7.3 目标函数与实际结算

日前优化目标为

$$\min C_d^{(2)} = \sum_t p_t G_{d,t} + \sum_{\omega} q_{d,\omega} \sum_t 5p_t R_{d,t}^{\omega}. \quad (11)$$

冻结计划后揭示实际净负荷，实际紧急购电和弃电分别为

$$R_{d,t}^{\text{act}} = \max\{n_{d,t}^{\text{act}} + C_{d,t} - G_{d,t} - D_{d,t}, 0\}, \quad (12)$$

$$W_{d,t}^{\text{act}} = \max\{G_{d,t} + D_{d,t} - C_{d,t} - n_{d,t}^{\text{act}}, 0\}. \quad (13)$$

正式期结果：计划购电费用为 **13 750 414.64** 元，紧急购电费用为 **1 507 608.25** 元，实际总费用为 **15 258 022.89** 元；紧急购电量为 **404 258.73 kWh**。预测 WAPE 为 **10.95%**。指定四日曲线见图 7-1。

表 7-1 指定日期、指定时段的购电量及全天汇总

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量/kWh
2025-03-20	0.00	805.09	0.00	458.78	756.58	0.00	73 287.0
2025-06-21	0.00	0.00	0.00	125.03	446.17	0.00	44 714.0
2025-09-23	0.00	716.69	0.00	382.61	836.37	9.31	72 727.1
2025-12-21	0.00	1 096.15	0.00	853.43	712.22	0.00	98 305.4

表 7-2 指定日期储能分段充/放电量（单元格为充电量/放电量，kWh）

日期	0-4 时	4-8 时	8-12 时	12-16 时	16-20 时	20-24 时	SOC(0 时/24 时)
2025-03-20	1666.67/0.00	1666.67/4352.03	33.33/473 158	93.33/104 133	0/5645.40	335.30/2994.66	0.00/5101.77
2025-06-21	462.26/0.00	833.33/2472.90	15.46/0.00	3285.30/38.70	0.00/5867.15	8166.67/27724.05	3.95/8550.00
2025-09-23	1666.67/0.00	833.33/6198.63	33.33/244 150	31.89/524.86	0/5809.57	8166.67/28308.33	0.00/8550.00
2025-12-21	1666.67/0.00	1666.67/1508.83	33.33/7806.63	33.33/2833.00	0/5733.07	8166.67/2906.93	0.00/8550.00

表 7-3 指定日期紧急购电

8 问题三：光伏预测更新下的滚动调整

8.1 为什么采用 MPC

在 0:00、6:00、12:00、18:00，附件 3 发布新的未来 24 小时光伏预测。新信息只影响尚未执行时段，过去的购电、充放电和费用都不可回写。这正对应“当前状态固定—

2025-03-20 紧急购电时间段	购电量/kWh
0:30-0:40	3.63
1:10-1:50	71.80
3:20-3:40	47.00
3:50-4:00	8.59
8:20-8:40	63.01
15:50-16:20	81.81
18:50-19:10	38.84
20:20-20:30	2.60
21:20-21:50	12.20
23:50-0:00+1	7.34

2025-06-21 紧急购电时间段	购电量/kWh
0:30-0:40	12.70
1:30-1:40	21.30
2:00-2:10	23.40
3:00-3:40	30.30
4:10-4:30	34.64
5:50-6:30	47.48
21:30-21:40	26.72

2025-09-23 紧急购电时间段	购电量/kWh
0:50-1:00	14.23
2:20-2:30	7.79
5:00-5:20	13.89
6:20-7:10	53.79
9:40-10:30	276.22
15:00-15:30	60.74
16:00-17:00	440.20
17:10-17:30	35.54
18:20-18:30	31.25
20:20-20:30	35.09
21:00-21:10	33.06
21:30-21:40	5.55
22:30-22:40	12.58

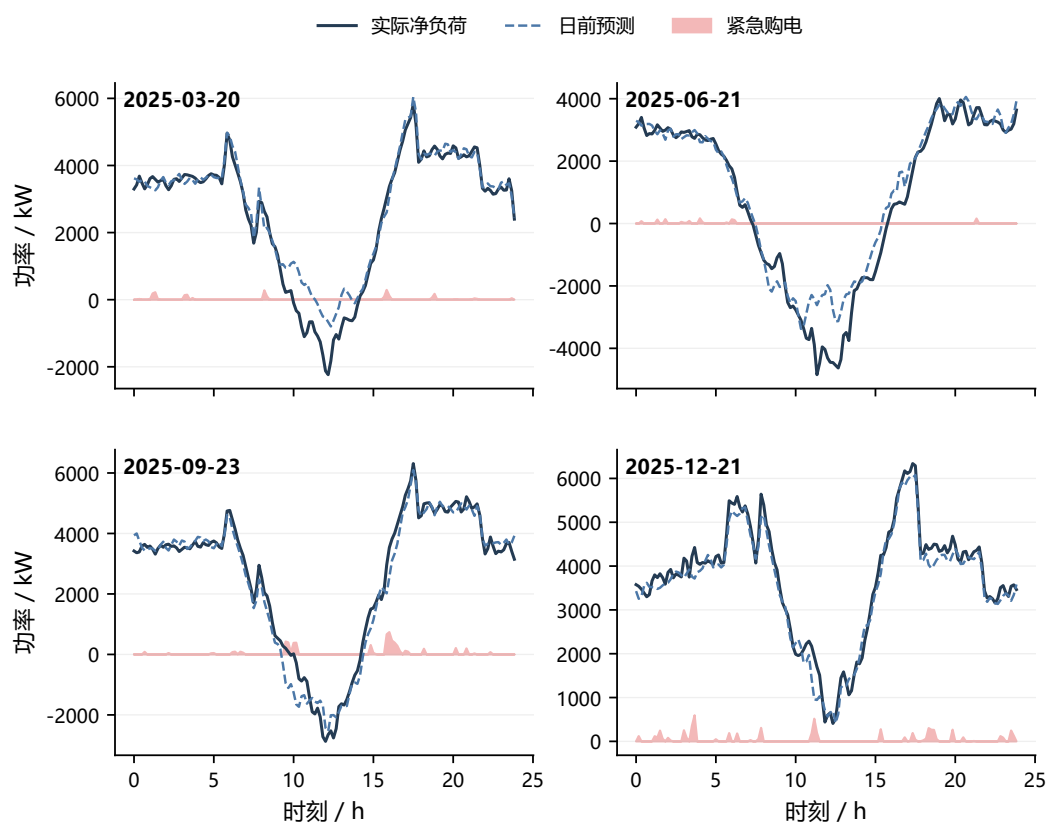


图 7-1 四个代表日的日前预测、实际净负荷与紧急购电

2025-12-21 紧急购电时间段	购电量/kWh
0:20-0:30	19.87
1:20-2:00	84.78
2:10-2:30	16.41
3:10-3:30	55.57
3:40-4:00	158.15
5:10-5:30	8.36
6:00-6:10	31.39
6:30-6:40	30.04
7:20-7:30	4.86
7:50-8:10	60.03
11:10-11:40	149.05
15:30-15:40	46.14
17:00-17:10	15.34
17:20-17:50	39.11
18:00-19:10	160.64
20:00-20:10	44.42
20:40-20:50	15.34
21:20-21:30	1.37
23:00-23:20	33.82
23:40-0:00+1	62.62

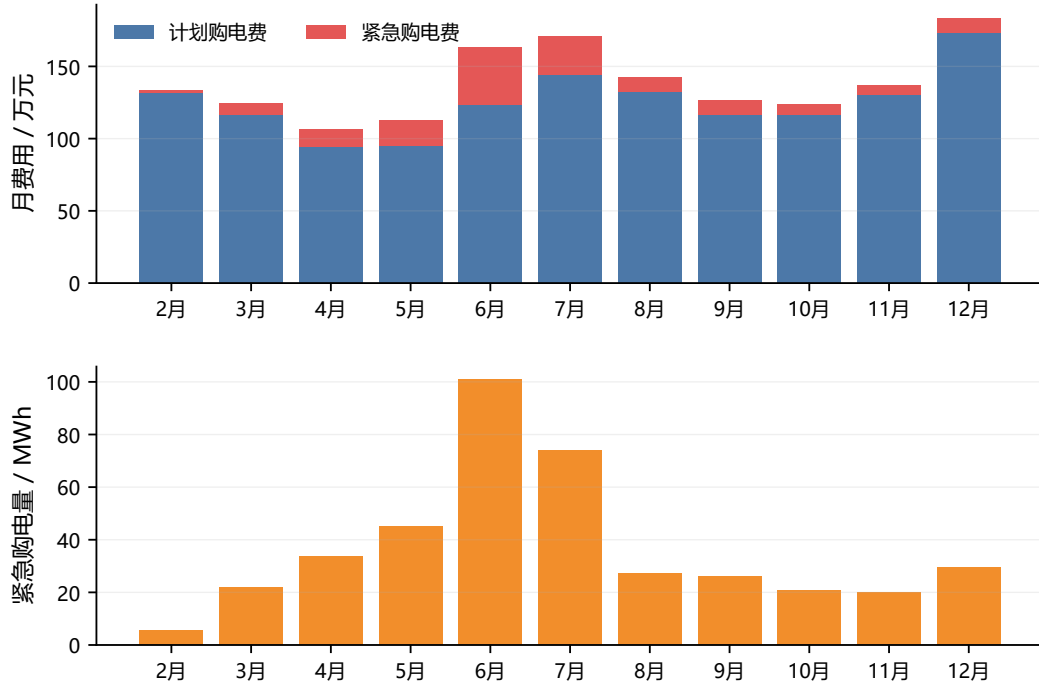


图 7-2 正式期月度费用分解与紧急购电量

在剩余时域重优化一只执行至下一次更新”的滚动结构。类似的日前一日内衔接和滚动时域思想已用于微网与局部能源系统调度 [2,4-6]，但本文的结算式和更新时刻均由本题单独推导。

附件 3 的整点预测以发布时刻已经观测的光伏为 0 小时锚点，分段线性插值到 10 分钟网格。未来负荷采用历史同星期曲线加日内已实现偏差的稳健校正，所有参数仍按决策日前滚动选择。

8.2 非对称调整费用

记 $\bar{G}_{d,t}^{(h-)}$ 为时刻 h 调整前的有效计划， $G_{d,t}^{(h)}$ 为新计划。定义

$$\Delta G_{d,t}^{+,h} = \max\{G_{d,t}^{(h)} - \bar{G}_{d,t}^{(h-)}, 0\}, \quad (4)$$

$$\Delta G_{d,t}^{-,h} = \max\{\bar{G}_{d,t}^{(h-)} - G_{d,t}^{(h)}, 0\}. \quad (14)$$

每次调整相对上一版有效计划逐次结算：

$$C_d^{\text{adj},h} = \sum_{t \geq h} (1.5p_t \Delta G_{d,t}^{+,h} - 0.5p_t \Delta G_{d,t}^{-,h}). \quad (15)$$

减购项前的负号表示从已经计入的原计划费用中退回减购量正常价格的 50%，不是另加 50% 罚款。实际总费用为

$$C_d^{(3)} = \sum_t p_t G_{d,t}^{(0)} + \sum_{h \in \{6,12,18\}} C_d^{\text{adj},h} + \sum_t 5p_t R_{d,t}^{\text{act}}. \quad (16)$$

8.3 更新频率消融与风险增强去留

更新集合	总费用/元	调整费用/元	紧急购电量/kWh	求解时间/s
仅 0:00	16 391 391.76	0	1 040 382.80	76.04
0:00、6:00	15 926 257.20	322 397.90	841 482.17	119.31
0:00、6:00、12:00	15 753 473.50	436 298.01	767 041.40	147.07
四次全部更新	15 731 687.04	499 519.56	748 796.98	133.33

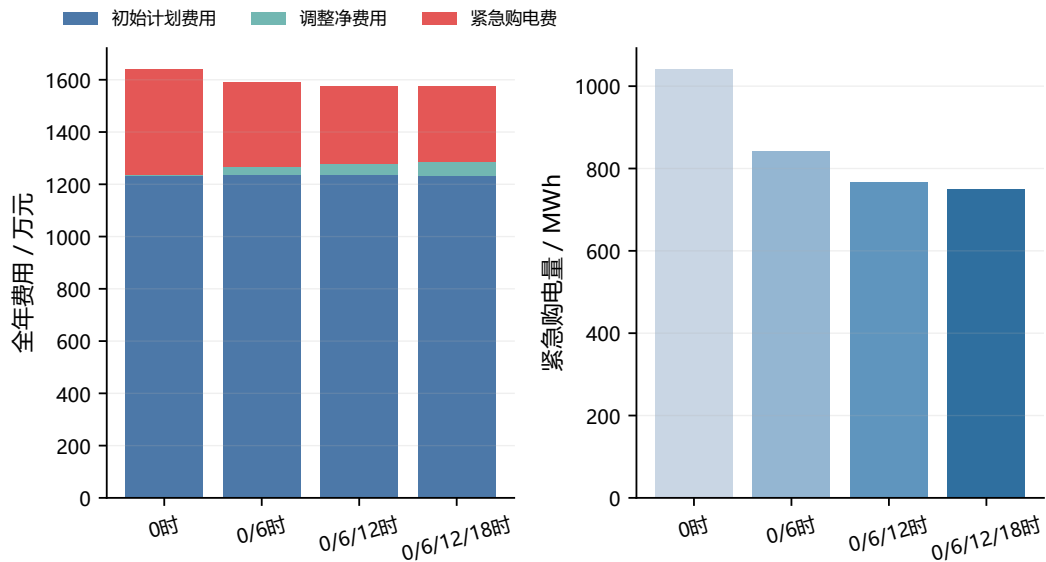


图 8-1 不同日内更新频率的费用与紧急购电量对比

表 8-2 指定日期、指定时段的 0 时计划量/最终有效量及全天汇总

表 8-3 指定日期储能分段充/放电量（单元格为充电量/放电量，kWh）

表 8-4 指定日期紧急购电

全年结果支持采用 0:00、6:00、12:00 与 18:00。结论不是“预报越新越一定要调整”：只有新增预报降低的紧急购电费用超过增减计划的交易成本时，该次更新才有净收益。场景 MPC 与 CVaR-MPC 涉及场景数量、概率、置信水平、风险权重和误差抽样方式。现有单年样本不足以同时客观识别这些参数，尚未满足进入九指标对照的前置条件；本文遵循预先锁定的简约门禁，不把风险增强写入主模型，也不虚构一组主观参数作形式化比较。

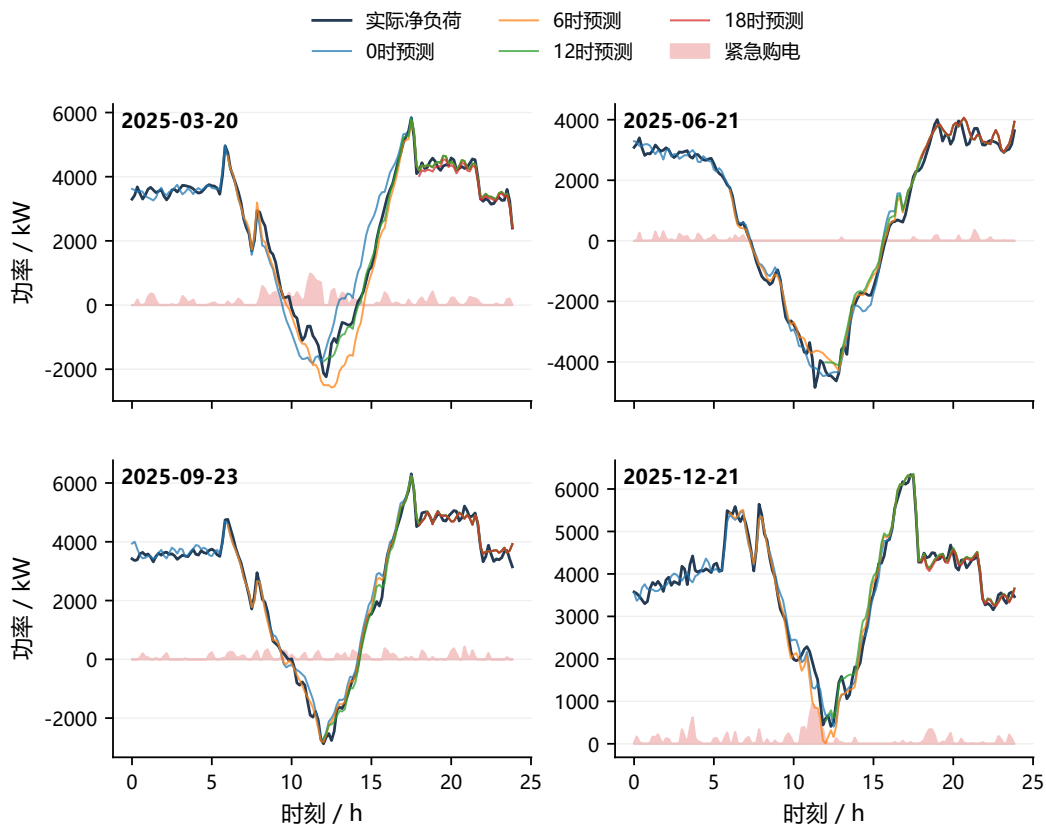


图 8-2 四个指定日的滚动预测修正效果

日期	10:00-10:10	12:00-12:10	14:00-14:10	16:00-16:10	18:00-18:10	20:00-20:10	全天购电量/kWh	全天购电费/元
2025-03-20	0.00/0.00	38.61/40.00	10620.00/10620.00	587.79/4713.27	7250.00/7250.00	0.00	64 162.72	55 675.48
2025-06-21	0.00/0.00	0.00/0.00	0.00/0.00	112.35/69.17	17.02/49.00	10.00	34 242.49	22 452.33
2025-09-23	0.00/0.00	82.08/3690.60	60.00/494.89	480.55/8050.00	0.00	0.00	66 693.08	50 514.40
2025-12-21	0.00/0.00	81.28/8070.00	420.00/96.14	815.38/7080.00	0.00	0.00	93 735.09	68 375.78

日期	0-4 时	4-8 时	8-12 时	12-16 时	16-20 时	20-24 时	SOC(0 时/24 时)
2025-03-20	1666.67/0.00	833.33/6429.33	28.27/22106.59	649.67/260.00	300/5806.97	8166.67/28336.00	8550.00/8550.00
2025-06-21	311.23/0.00	833.33/1772.63	75.61/0.00	9091.06/0.00	33.57/6089.95	355.08/2572.13	39.47/2419.58
2025-09-23	1666.67/0.00	833.33/6860.54	33.73/1775.83	33.97/324.84	400/5654.80	8166.67/29833.00	8550.00/8550.00
2025-12-21	1666.67/0.00	1661.40/1565.83	33.33/7469.64	2.90/255.00	700/5739.72	8166.67/29083.00	8550.00/8550.00

2025-03-20 紧急购电时间段	购电量/kWh
0:30-0:40	28.15
1:00-1:50	188.73
2:40-2:50	3.74
3:10-4:10	146.15
5:00-5:40	31.18
6:10-6:30	21.32
6:40-7:10	68.39
8:10-9:30	486.05
9:40-12:10	1 150.75
12:40-14:40	461.35
15:30-17:00	198.89
17:30-17:50	32.16
18:10-19:20	239.07
20:00-20:50	106.46
21:10-22:10	161.52
23:00-23:10	8.29
23:40-0:10+1	63.06

2025-06-21 紧急购电时间段	购电量/kWh
0:30-0:40	44.34
1:20-2:20	142.23
2:30-2:40	9.36
3:00-3:40	110.46
4:10-4:30	64.53
4:50-5:30	69.31
6:10-6:20	14.90
6:30-7:30	93.37
13:10-13:20	18.54
17:40-18:00	29.32
19:00-19:20	57.43
19:50-20:00	0.60
20:30-20:40	29.16
21:20-21:50	112.68
22:20-22:40	26.43
23:20-23:30	1.30

2025-09-23 紧急购电时间段	购电量/kWh
0:40-1:20	68.77
1:40-1:50	7.87
2:10-2:30	52.90
3:30-3:40	5.66
5:00-5:30	74.13
5:50-7:30	194.83
7:50-8:10	76.65
8:30-9:00	125.50
9:30-10:30	144.49
10:50-11:10	58.63
11:40-12:10	87.67
13:00-15:10	331.68
16:00-16:50	185.54
17:10-17:30	15.62
17:40-18:00	15.02
18:10-18:50	88.20
19:00-19:10	12.45
19:30-19:40	2.71
20:00-20:40	150.89
21:00-21:50	166.43
22:00-22:10	1.05
22:30-22:40	34.92
23:40-23:50	12.31

2025-12-21 紧急购电时间段	购电量/kWh
0:10-0:40	38.86
1:10-2:00	106.07
2:10-2:30	40.84
3:10-3:30	86.11
3:40-4:30	201.58
5:10-5:40	47.20
5:50-6:10	65.84
6:20-6:40	53.68
7:10-7:40	49.56
7:50-8:20	69.97
8:30-8:40	17.65
9:00-9:10	12.74
9:40-10:10	92.39
10:40-12:20	818.26
12:50-13:20	35.37
14:00-14:10	23.53
16:30-16:40	7.02
17:00-17:10	11.98
17:30-17:40	7.18
18:10-19:10	208.44
19:30-20:10	80.18
20:40-21:00	24.26
21:20-21:30	1.08
21:50-22:10	6.73
22:50-23:20	51.71
23:40-0:00+1	56.33

9 问题四：实时价格下的因果策略与信息价值

9.1 因果价格预测

附件 4 给出的是全年实现价格，而不是每日 0:00 已知的未来报价。本文比较 1 日—7 日季节加权法与“历史同星期中位曲线 + 日内前缀偏差校正”，只用预测原点以前的价格选择参数。Q4-2 每日 0:00 预测全天价格并求解严格两阶段模型；Q4-3 在四个发布时刻同步更新价格与光伏信息，并滚动求解未执行时段。

9.2 两类评价基准

因果主策略满足现实信息边界。为了评价信息价值，另计算：

1. 完全价格信息反事实基准：只把未来实时价格交给优化器，负荷和光伏信息仍与主策略一致。由于预测误差仍存在，它主要度量价格预测损失；
2. 全外生信息 **oracle** 严格下界：一次性给出全年价格、负荷和光伏实际路径，并对充放电互斥变量作连续松弛。它同时放宽信息限制和整数约束，故目标值严格构成现实策略成本的下界，但不是可执行方案。

策略	Q4-2 总费用/元	Q4-3 总费用/元	角色
因果在线策略	16 124 373.46	16 550 967.30	正式结果
完全价格信息基准	15 923 572.41	16 401 924.52	价格信息评价
全信息 oracle	12 782 731.61	12 782 731.61	严格下界

表 9-2 波动电价下指定日期结果

日期	Q4-2 总费用/元	Q4-2 紧急购电/kWh	Q4-3 总费用/元	Q4-3 紧急购电/kWh
2025-03-20	49 580.79	336.20	57 894.95	3 395.27
2025-06-21	20 454.56	196.52	19 433.16	823.95
2025-09-23	50 611.49	1 019.05	51 825.88	1 913.89
2025-12-21	78 821.59	1 037.31	80 925.21	2 214.58

价格信息差额分别为 **200 801.05** 元和 **149 042.78** 元。结果表明在本样本中日内调整交易成本抵消了信息更新收益，日前因果策略更经济。

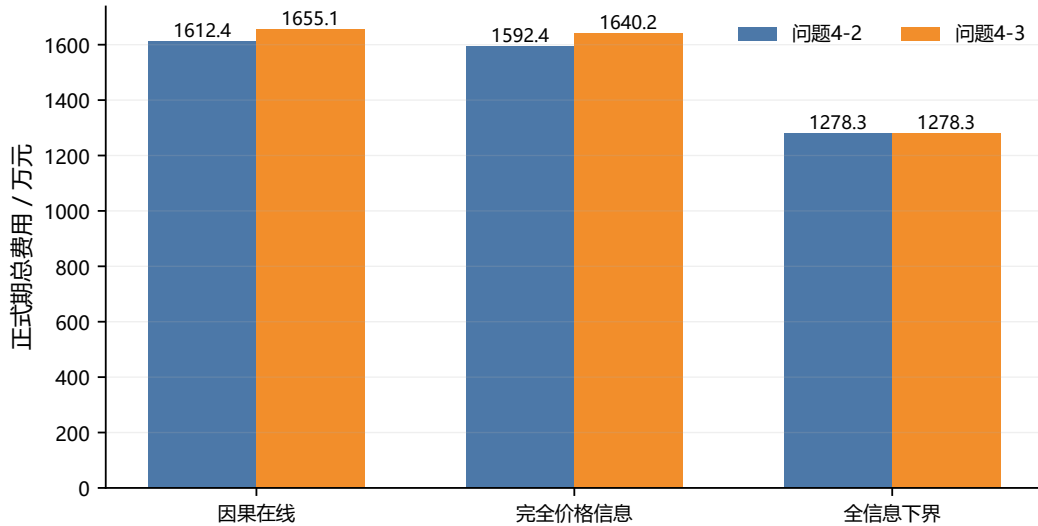


图 9-1 因果策略、完全价格信息基准与全信息下界的费用对比

10 模型检验、灵敏度与结果可靠性

10.1 硬约束与费用账本检验

所有问均逐时段复算式 (1) 和式 (2)，检查 E 位于 $[1200, 10800]$ kWh，充放电互斥，跨日满足式 (5)。Q3/Q4 还逐版本复算式 (15)，验证调整费用始终相对上一版有效计划。各问最大残差均小于 $8.20\text{e-}12$ kWh。

10.2 时间信息泄漏检验

程序为每个预测原点保存训练截止日期，断言训练标签早于预测原点；标准化尺度、历史分位数、相似日距离、残差库和超参数均由过去数据估计。附件 2、4 当天未来实际值只在计划冻结后进入结算。完全信息基准使用单独函数与输出前缀，避免混入正式结果。

10.3 模型对照

Q1 使用 LP 连续松弛和无储能基线；Q2 报告滚动样本外 WAPE、紧急购电量与费用分解；Q3 比较四种更新频率；Q4 比较因果策略、完全价格信息基准和 oracle 下界。除 oracle 为保证下界性质而明确放松信息与整数约束外，其余比较保持储能容量、效率、功率与实际结算路径一致。

11 模型评价

11.1 优点

1. 模型结构由题目中的物理关系、信息到达和结算规则逐层产生，四问共享同一能量与 SOC 骨架。
2. Q2 明确区分 0:00 固定计划和实际紧急追索，Q3 明确冻结已执行时段，避免未来数据泄漏与决策回写。
3. 采用 MILP 全局求解并保存残差、费用账本和输入哈希，关键数字可从论文追溯到结果 JSON、代码和原始附件。
4. 风险增强采用“结果显著改善才保留”的门禁，避免机械叠加算法。

11.2 局限与改进

1. 只有一年历史数据，极端天气和跨年季节变化的覆盖有限；若获得多年气象数据，可进一步建立条件光伏误差模型。
2. 两阶段情景由历史整日残差构造，能够保留日内相关，但对从未出现过的极端路径刻画不足。
3. 未计电池寿命衰减、启停损耗和需量电费，因为题目未给出相应参数；工程应用中应在有可靠参数后加入循环寿命成本。
4. 全信息 oracle 仅用于评价，不代表现实可达到的费用。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于赛题理解辅助、候选模型比较、代码编写与调试、结果核验、图表生成和文字润色，详细使用情况见支撑材料。

参考文献

[1] 张亚琳. 具有电池储能系统的微电网分布式经济调度方案与 SoC 均衡控制 [D]. 天津: 南开大学, 2025. DOI:10.27254/d.cnki.gnkau.2025.000023. [2] 李嘉伟, 巨云涛, 张璐, 等. 基于分布鲁棒模型预测控制的微电网多时间尺度优化调度 [J]. 电力工程技术, 2024, 43(4):45-55. DOI:10.12158/j.2096-3203.2024.04.005. [3] 张理, 王宝, 贾健雄, 等. 微电网功率预测与调度端到端协同优化方法 [J]. 上海交通大学学报, 2025, 59(6):720-731.

DOI:10.16183/j.cnki.jsjtu.2024.224. [4] LIU C, QIN Y, ZHANG H. Real-time scheduling strategy for microgrids considering operation interval division of DGs and batteries[J]. Global Energy Interconnection, 2020, 3(5):442-452. DOI:10.1016/j.gloi.2020.11.004. [5] OROZCO C, BORGHETTI A, DE SCHUTTER B, et al. Intra-day scheduling of a local energy community coordinated with day-ahead multistage decisions[J]. Sustainable Energy, Grids and Networks, 2022, 29:100573. DOI:10.1016/j.segan.2021.100573. [6] ELKAZAZ M, SUMNER M, THOMAS D. Energy management system for hybrid PV-wind-battery microgrid using convex programming, model predictive and rolling horizon predictive control with experimental validation[J]. International Journal of Electrical Power & Energy Systems, 2020, 115:105483. DOI:10.1016/j.ijepes.2019.105483. [7] 侯慧, 王晴, 薛梦雅, 等. 计及源荷不确定性及需求响应的离网型微电网两阶段日前经济调度 [J]. 电力系统保护与控制, 2022, 50(13):73-85. DOI:10.19783/j.cnki.pspc.211406. [8] 刘一欣, 郭力, 王成山. 微电网两阶段鲁棒优化经济调度方法 [J]. 中国电机工程学报, 2018, 38(14):4013-4022, 4307. DOI:10.13334/j.0258-8013.pcsee.170500. [9] 王灿, 张雪菲, 凌凯, 等. 基于区间概率不确定集的微电网两阶段自适应鲁棒优化调度 [J]. 中国电机工程学报, 2024, 44(5):1750-1764. DOI:10.13334/j.0258-8013.pcsee.221968.

A 完整源程序代码

本附录列出全部可运行源程序；同名文件同时收入支撑材料。

A.1 common_milp.py

```
1  """微网调度的公共混合整数线性规划骨架。
2
3  变量统一使用每个离散时段内的电量（kWh）。储能充电量为交流母线侧
4  输入电量，放电量为交流母线侧输出电量，因此状态转移为
5   $E[t+1] = E[t] + \eta_c \cdot \text{charge}[t] - \text{discharge}[t] / \eta_d$ 。
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import dataclass
11 from time import perf_counter
12
13 import numpy as np
14 from scipy.optimize import Bounds, LinearConstraint, milp
15 from scipy.sparse import lil_matrix
16
17
18 @dataclass
19 class DispatchResult:
20     grid: np.ndarray
21     charge: np.ndarray
22     discharge: np.ndarray
23     curtailment: np.ndarray
24     emergency: np.ndarray
25     soc: np.ndarray
26     mode: np.ndarray
27     objective: float
28     solve_seconds: float
29     max_balance_residual: float
30     max_soc_residual: float
31     success: bool
32     message: str
33
34
35 def solve_deterministic_dispatch(
36     load_kwh: np.ndarray,
37     pv_kwh: np.ndarray,
```

```

38     price: np.ndarray,
39     initial_soc: float,
40     *,
41     terminal_soc: float | None = None,
42     eta_c: float = 0.9,
43     eta_d: float = 0.9,
44     soc_min: float = 1200.0,
45     soc_max: float = 10800.0,
46     max_power_kw: float = 5000.0,
47     step_hours: float = 1.0 / 6.0,
48     allow_emergency: bool = False,
49     emergency_multiplier: float = 5.0,
50     binary_mode: bool = True,
51     time_limit: float = 120.0,
52 ) -> DispatchResult:
53     """求解一个确定性时域内的购电与储能联合调度。"""
54
55     load = np.asarray(load_kwh, dtype=float)
56     pv = np.asarray(pv_kwh, dtype=float)
57     tariff = np.asarray(price, dtype=float)
58     if not (load.ndim == pv.ndim == tariff.ndim == 1):
59         raise ValueError("load_kwh、pv_kwh和price必须是一维数组")
60     if not (len(load) == len(pv) == len(tariff)):
61         raise ValueError("负荷、光伏和电价长度必须一致")
62     if np.any(load < 0) or np.any(pv < 0) or np.any(tariff < 0):
63         raise ValueError("负荷、光伏和电价不得为负")
64
65     n = len(load)
66     # g, c, d, q(弃光), r(紧急购电), E[0:n], z
67     g = slice(0, n)
68     c = slice(n, 2 * n)
69     d = slice(2 * n, 3 * n)
70     q = slice(3 * n, 4 * n)
71     r = slice(4 * n, 5 * n)
72     e = slice(5 * n, 6 * n + 1)
73     z = slice(6 * n + 1, 7 * n + 1)
74     nvar = 7 * n + 1
75
76     obj = np.zeros(nvar)
77     obj[g] = tariff
78     obj[r] = emergency_multiplier * tariff
79
80     lower = np.zeros(nvar)

```

```

81 upper = np.full(nvar, np.inf)
82 interval_limit = max_power_kw * step_hours
83 upper[c] = interval_limit
84 upper[d] = interval_limit
85 upper[q] = pv
86 upper[r] = np.inf if allow_emergency else 0.0
87 lower[e], upper[e] = soc_min, soc_max
88 lower[5 * n] = upper[5 * n] = initial_soc
89 if terminal_soc is not None:
90     lower[6 * n] = upper[6 * n] = terminal_soc
91 upper[z] = 1.0
92
93 # 供需平衡和SOC状态转移。
94 aeq = lil_matrix((2 * n, nvar), dtype=float)
95 beq = np.zeros(2 * n)
96 for t in range(n):
97     aeq[t, g.start + t] = 1.0
98     aeq[t, c.start + t] = -1.0
99     aeq[t, d.start + t] = 1.0
100    aeq[t, q.start + t] = -1.0
101    aeq[t, r.start + t] = 1.0
102    beq[t] = load[t] - pv[t]
103
104    row = n + t
105    aeq[row, e.start + t] = -1.0
106    aeq[row, e.start + t + 1] = 1.0
107    aeq[row, c.start + t] = -eta_c
108    aeq[row, d.start + t] = 1.0 / eta_d
109
110 # c_t <= M z_t, d_t <= M(1-z_t)。LP对照时z连续。
111 aub = lil_matrix((2 * n, nvar), dtype=float)
112 bub = np.zeros(2 * n)
113 for t in range(n):
114     aub[t, c.start + t] = 1.0
115     aub[t, z.start + t] = -interval_limit
116     aub[n + t, d.start + t] = 1.0
117     aub[n + t, z.start + t] = interval_limit
118     bub[n + t] = interval_limit
119
120 integrality = np.zeros(nvar, dtype=int)
121 if binary_mode:
122     integrality[z] = 1
123

```

```

124     constraints = [
125         LinearConstraint(aeq.tocsr(), beq, beq),
126         LinearConstraint(aub.tocsr(), -np.inf, bub),
127     ]
128     started = perf_counter()
129     raw = milp(
130         obj,
131         integrality=integrality,
132         bounds=Bounds(lower, upper),
133         constraints=constraints,
134         options={"time_limit": time_limit, "mip_rel_gap": 1e-8},
135     )
136     elapsed = perf_counter() - started
137     if raw.x is None:
138         raise RuntimeError(f"MILP求解失败: {raw.message}")
139
140     x = raw.x
141     grid = x[g]
142     charge = x[c]
143     discharge = x[d]
144     curtailment = x[q]
145     emergency = x[r]
146     soc = x[e]
147     mode = x[z]
148     balance_residual = grid + pv + discharge + emergency - load - charge - curtailment
149     soc_residual = soc[1:] - soc[:-1] - eta_c * charge + discharge / eta_d
150     return DispatchResult(
151         grid=grid,
152         charge=charge,
153         discharge=discharge,
154         curtailment=curtailment,
155         emergency=emergency,
156         soc=soc,
157         mode=mode,
158         objective=float(raw.fun),
159         solve_seconds=elapsed,
160         max_balance_residual=float(np.max(np.abs(balance_residual))),
161         max_soc_residual=float(np.max(np.abs(soc_residual))),
162         success=bool(raw.success),
163         message=str(raw.message),
164     )

```

A.2 q1_model.py

```
1  """问题一正式求解：确定性MILP主模型与LP连续松弛对照。"""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11  from common_milp import solve_deterministic_dispatch
12  from data_loader import ROOT, load_q1
13
14
15  def _summary(result) -> dict:
16      return {
17          "objective_yuan": float(result.objective),
18          "grid_kwh": float(result.grid.sum()),
19          "charge_kwh": float(result.charge.sum()),
20          "discharge_kwh": float(result.discharge.sum()),
21          "curtailment_kwh": float(result.curtailment.sum()),
22          "soc_start_kwh": float(result.soc[0]),
23          "soc_end_kwh": float(result.soc[-1]),
24          "soc_min_kwh": float(result.soc.min()),
25          "soc_max_kwh": float(result.soc.max()),
26          "simultaneous_intervals": int(np.sum((result.charge > 1e-7) & (result.discharge > 1e-7)
27          )),
28          "max_balance_residual_kwh": float(result.max_balance_residual),
29          "max_soc_residual_kwh": float(result.max_soc_residual),
30          "solve_seconds": float(result.solve_seconds),
31          "success": bool(result.success),
32          "solver_message": result.message,
33      }
34
35  def main() -> None:
36      data = load_q1()
37      kwargs = dict(
38          load_kwh=data["load_kwh"], pv_kwh=data["pv_kwh"], price=data["price"],
39          initial_soc=6000.0, terminal_soc=6000.0, allow_emergency=False,
40      )
```

```

41 milp_result = solve_deterministic_dispatch(**kwargs, binary_mode=True)
42 lp_result = solve_deterministic_dispatch(**kwargs, binary_mode=False)
43
44 selected_labels = ["10:00-10:10", "12:00-12:10", "14:00-14:10", "16:00-16:10",
45 "18:00-18:10", "20:00-20:10"]
46 # 同序号对齐下, 10:00-10:10对应附件第60项(数组下标59), 其后每2小时增加12项。
47 selected_indices = [59, 71, 83, 95, 107, 119]
48 selected = {
49     label: float(milp_result.grid[idx])
50     for label, idx in zip(selected_labels, selected_indices)
51 }
52 blocks = []
53 for start in range(0, 144, 24):
54     blocks.append({
55         "interval": f"{start // 6}:00-{(start + 24) // 6}:00",
56         "charge_kwh": float(milp_result.charge[start:start + 24].sum()),
57         "discharge_kwh": float(milp_result.discharge[start:start + 24].sum()),
58         "soc_end_kwh": float(milp_result.soc[start + 24]),
59     })
60 no_storage_cost = float(np.dot(data["price"], np.maximum(data["load_kwh"] - data["pv_kwh"],
61 0.0)))
62 report = {
63     "question": "Q1",
64     "model": "deterministic_MILP_with_charge_discharge_mutex",
65     "status": "success",
66     "source_files": ["problems/选题C_2026正式/附件/附件1.xlsx"],
67     "time_step_hours": 1.0 / 6.0,
68     "alignment_rule": "附件与结果模板按同序号对应, 不循环平移",
69     "milp": _summary(milp_result),
70     "lp_relaxation": _summary(lp_result),
71     "lp_lower_bound_gap_yuan": float(milp_result.objective - lp_result.objective),
72     "no_storage_baseline_cost_yuan": no_storage_cost,
73     "cost_saving_vs_no_storage_yuan": float(no_storage_cost - milp_result.objective),
74     "cost_saving_rate": float((no_storage_cost - milp_result.objective) / no_storage_cost),
75     "selected_interval_grid_kwh": selected,
76     "four_hour_blocks": blocks,
77     "validation": {
78         "balance_residual_le_1e-6": bool(milp_result.max_balance_residual <= 1e-6),
79         "soc_residual_le_1e-6": bool(milp_result.max_soc_residual <= 1e-6),
80         "soc_bounds": bool(milp_result.soc.min() >= 1200 - 1e-6 and milp_result.soc.max()
81 <= 10800 + 1e-6),
82         "terminal_soc": bool(abs(milp_result.soc[-1] - 6000) <= 1e-6),

```

```

81         "no_simultaneous": bool(np.all((milp_result.charge <= 1e-7) | (milp_result.
discharge <= 1e-7))),
82         "lp_lower_bound": bool(lp_result.objective <= milp_result.objective + 1e-6),
83     },
84     "claims": [
85         {
86             "claim_id": "Q1-COST",
87             "value": float(milp_result.objective),
88             "unit": "元/日",
89             "code": "code/q1_model.py",
90             "data": "problems/选题C_2026正式/附件/附件1.xlsx",
91         }
92     ],
93 }
94 report["status"] = "success" if all(report["validation"].values()) else "failed"
95
96 out_dir = ROOT / "results"
97 out_dir.mkdir(exist_ok=True)
98 details = pd.DataFrame({
99     "slot": np.arange(1, 145), "time": data["time"], "price_yuan_per_kwh": data["price"],
100     "load_kwh": data["load_kwh"], "pv_kwh": data["pv_kwh"], "grid_kwh": milp_result.grid,
101     "charge_kwh": milp_result.charge, "discharge_kwh": milp_result.discharge,
102     "curtailment_kwh": milp_result.curtailment, "soc_start_kwh": milp_result.soc[:-1],
103     "soc_end_kwh": milp_result.soc[1:],
104 })
105 details.to_csv(out_dir / "q1_detail.csv", index=False, encoding="utf-8-sig")
106 result_path = ROOT / "state" / "agent_outputs" / "q1_results.json"
107 result_path.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
108 print(json.dumps(report, ensure_ascii=False, indent=2))
109
110
111 if __name__ == "__main__":
112     main()

```

A.3 formal_data_loader.py

```

1  """2026 C题正式数据读取器。
2
3  功能：读取附件1、附件2并执行结构、缺失和单位检查。
4  输入：problems/选题C_2026正式/附件/附件1.xlsx、附件2.xlsx
5  输出：内存中的日×时段数组；功率同时换算为10分钟电量。
6  运行：由 q2_model.py 导入。
7  """

```

```

8
9 from __future__ import annotations
10
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15
16
17 ROOT = Path(__file__).resolve().parents[1]
18 DATA_DIR = ROOT / "problems" / "选题C_2026正式" / "附件"
19 STEP_HOURS = 1.0 / 6.0
20 N_DAYS = 365
21 N_STEPS = 144
22
23
24 def _read_daily_sheet(path: Path, sheet_name: str) -> tuple[np.ndarray, np.ndarray]:
25     """读取附件2的一个365×144宽表，返回日期与数值矩阵。"""
26
27     frame = pd.read_excel(path, sheet_name=sheet_name)
28     if frame.shape != (N_DAYS, N_STEPS + 1):
29         raise ValueError(
30             f"{path.name}/{sheet_name}形状应为(365,145)，实际为{frame.shape}"
31         )
32     if frame.iloc[:, 1:].isna().any().any():
33         raise ValueError(f"{path.name}/{sheet_name}存在缺失数值")
34     dates = pd.to_datetime(frame.iloc[:, 0], errors="raise").to_numpy()
35     expected = pd.date_range("2025-01-01", "2025-12-31", freq="D").to_numpy()
36     if not np.array_equal(dates.astype("datetime64[D]"), expected.astype("datetime64[D]")):
37         raise ValueError(f"{path.name}/{sheet_name}日期不连续或不是2025全年")
38     values = frame.iloc[:, 1:].to_numpy(dtype=float)
39     if not np.isfinite(values).all() or np.any(values < 0):
40         raise ValueError(f"{path.name}/{sheet_name}含非有限值或负功率")
41     return dates, values
42
43
44 def load_typical_day() -> dict[str, np.ndarray]:
45     """读取附件1的固定电价与典型日曲线。"""
46
47     path = DATA_DIR / "附件1.xlsx"
48     frame = pd.read_excel(path)
49     expected_columns = ["时间", "电价", "小区负载", "光伏发电预测功率"]
50     if list(frame.columns) != expected_columns or frame.shape[0] != N_STEPS:

```

```

51         raise ValueError("附件1列结构或时段数异常")
52     if frame.isna().any().any():
53         raise ValueError("附件1存在缺失值")
54     price = frame["电价"].to_numpy(dtype=float)
55     load_kw = frame["小区负载"].to_numpy(dtype=float)
56     pv_kw = frame["光伏发电预测功率"].to_numpy(dtype=float)
57     if np.any(price < 0) or np.any(load_kw < 0) or np.any(pv_kw < 0):
58         raise ValueError("附件1不应出现负的价格、负荷或光伏功率")
59     return {
60         "time": frame["时间"].astype(str).to_numpy(),
61         "price": price,
62         "load_kw": load_kw,
63         "pv_kw": pv_kw,
64         "load_kwh": load_kw * STEP_HOURS,
65         "pv_kwh": pv_kw * STEP_HOURS,
66         "net_kwh": (load_kw - pv_kw) * STEP_HOURS,
67     }
68
69
70 def load_q2_data() -> dict[str, np.ndarray]:
71     """读取Q2全年实际负荷/光伏，统一为kW与每时段kWh。"""
72
73     path = DATA_DIR / "附件2.xlsx"
74     dates_l, load_kw = _read_daily_sheet(path, "小区负载")
75     dates_s, pv_kw = _read_daily_sheet(path, "光伏发电实际功率")
76     if not np.array_equal(dates_l, dates_s):
77         raise ValueError("附件2两个工作表的日期不一致")
78     load_kwh = load_kw * STEP_HOURS
79     pv_kwh = pv_kw * STEP_HOURS
80     return {
81         "dates": dates_l.astype("datetime64[D]"),
82         "load_kw": load_kw,
83         "pv_kw": pv_kw,
84         "load_kwh": load_kwh,
85         "pv_kwh": pv_kwh,
86         "net_kwh": load_kwh - pv_kwh,
87     }

```

A.4 causal_forecasts.py

```

1  """Q2因果日前净负荷预测与整日残差情景生成。
2
3  所有函数均通过 history_end 明确限制标签可见范围。任何目标日的真实值只在

```

```

4  该日完成后进入残差库。候选包括季节加权朴素法N1与岭回归N3。
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass
10
11 import numpy as np
12 from sklearn.linear_model import Ridge
13 from sklearn.pipeline import make_pipeline
14 from sklearn.preprocessing import StandardScaler
15
16
17 ALPHA_GRID = (0.0, 0.25, 0.5, 0.75, 1.0)
18 RIDGE_LAMBDA_GRID = (0.01, 0.1, 1.0, 10.0, 100.0)
19 RIDGE_WINDOW_GRID = (56, 112, 10000)
20
21
22 @dataclass(frozen=True)
23 class ForecastChoice:
24     method: str
25     params: dict[str, float | int | str]
26     validation_wape: float
27     validation_mae: float
28     validation_days: int
29
30
31 def _safe_lag(net: np.ndarray, day: int, lag: int, fallback: np.ndarray) -> np.ndarray:
32     idx = day - lag
33     return net[idx] if idx >= 0 else fallback
34
35
36 def seasonal_forecast(
37     net: np.ndarray,
38     day: int,
39     alpha: float,
40     fallback: np.ndarray,
41     *,
42     recursive_lag1: np.ndarray | None = None,
43 ) -> np.ndarray:
44     """N1预测; day可以等于历史截止日以后的第一天。"""
45
46     if day <= 0:

```

```

47         return np.asarray(fallback, dtype=float).copy()
48     lag1 = recursive_lag1 if recursive_lag1 is not None else net[day - 1]
49     lag7 = _safe_lag(net, day, 7, lag1)
50     return alpha * np.asarray(lag1) + (1.0 - alpha) * np.asarray(lag7)
51
52
53 def _day_features(
54     net: np.ndarray,
55     load: np.ndarray,
56     pv: np.ndarray,
57     day: int,
58     fallback: np.ndarray,
59     *,
60     lag1_override: np.ndarray | None = None,
61 ) -> np.ndarray:
62     """构造某目标日144条记录；只引用目标日前的日数据。"""
63
64     t = np.arange(144, dtype=float)
65     doy = float(day + 1)
66     weekday = day % 7
67     lag1 = lag1_override if lag1_override is not None else _safe_lag(net, day, 1, fallback)
68     lag7 = _safe_lag(net, day, 7, lag1)
69     start = max(0, day - 7)
70     hist = net[start:day]
71     if hist.shape[0] == 0:
72         hist_mean = fallback
73         hist_std = np.zeros(144)
74     else:
75         hist_mean = hist.mean(axis=0)
76         hist_std = hist.std(axis=0)
77     prev_load = load[day - 1] if day >= 1 else np.maximum(fallback, 0.0)
78     prev_pv = pv[day - 1] if day >= 1 else np.maximum(-fallback, 0.0)
79     scalar = np.column_stack(
80         [
81             np.sin(2 * np.pi * t / 144),
82             np.cos(2 * np.pi * t / 144),
83             np.full(144, np.sin(2 * np.pi * doy / 365)),
84             np.full(144, np.cos(2 * np.pi * doy / 365)),
85             np.full(144, float(weekday >= 5)),
86             np.full(144, float(weekday)),
87             lag1,
88             lag7,
89             hist_mean,

```

```

90         hist_std,
91         np.full(144, float(np.mean(lag1))),
92         np.full(144, float(np.max(lag1))),
93         np.full(144, float(np.min(lag1))),
94         np.full(144, float(np.sum(prev_load))),
95         np.full(144, float(np.sum(prev_pv))),
96     ]
97 )
98 return scalar
99
100
101 def fit_ridge(
102     net: np.ndarray,
103     load: np.ndarray,
104     pv: np.ndarray,
105     history_end: int,
106     lam: float,
107     window: int,
108     fallback: np.ndarray,
109 ):
110     """在[0, history_end)日拟合N3，窗口和标准化均仅用过去。"""
111
112     train_start = max(1, history_end - int(window))
113     train_days = list(range(train_start, history_end))
114     if len(train_days) < 7:
115         return None
116
117     x = np.vstack([_day_features(net, load, pv, d, fallback) for d in train_days])
118     y = np.concatenate([net[d] for d in train_days])
119     model = make_pipeline(StandardScaler(), Ridge(alpha=float(lam), fit_intercept=True))
120     model.fit(x, y)
121     return model
122
123 def ridge_forecast(
124     model,
125     net: np.ndarray,
126     load: np.ndarray,
127     pv: np.ndarray,
128     day: int,
129     fallback: np.ndarray,
130     *,
131     lag1_override: np.ndarray | None = None,
132 ) -> np.ndarray:

```

```

133     if model is None:
134         return seasonal_forecast(net, day, 1.0, fallback, recursive_lag1=lag1_override)
135     x = _day_features(net, load, pv, day, fallback, lag1_override=lag1_override)
136     return np.asarray(model.predict(x), dtype=float)
137
138
139 def _metrics(actual: np.ndarray, predicted: np.ndarray) -> tuple[float, float]:
140     err = np.asarray(predicted) - np.asarray(actual)
141     mae = float(np.mean(np.abs(err)))
142     den = float(np.sum(np.abs(actual)))
143     wape = float(np.sum(np.abs(err)) / den) if den > 1e-12 else float("inf")
144     return wape, mae
145
146
147 def select_month_model(
148     net: np.ndarray,
149     load: np.ndarray,
150     pv: np.ndarray,
151     month_start: int,
152     fallback: np.ndarray,
153 ) -> ForecastChoice:
154     """用月首前最近14日作严格时间验证，选择N1或N3。
155
156     岭回归在验证段开始前一次拟合，因此其系数对14个验证原点均为过去信息；
157     滞后特征则按各验证日当时已完成的数据逐日构造。
158     """
159
160     val_start = max(1, month_start - 14)
161     val_days = list(range(val_start, month_start))
162     if not val_days:
163         return ForecastChoice("N1", {"alpha": 1.0}, float("nan"), float("nan"), 0)
164
165     candidates: list[tuple[float, float, int, str, dict[str, float | int | str]]] = []
166     for alpha in ALPHA_GRID:
167         preds = np.vstack([seasonal_forecast(net, d, alpha, fallback) for d in val_days])
168         wape, mae = _metrics(net[val_days], preds)
169         candidates.append((wape, mae, 1, "N1", {"alpha": alpha}))
170
171     train_end = val_start
172     for window in RIDGE_WINDOW_GRID:
173         for lam in RIDGE_LAMBDA_GRID:
174             model = fit_ridge(net, load, pv, train_end, lam, window, fallback)
175             if model is None:

```

```

176         continue
177     preds = np.vstack(
178         [ridge_forecast(model, net, load, pv, d, fallback) for d in val_days]
179     )
180     wape, mae = _metrics(net[val_days], preds)
181     candidates.append(
182         (wape, mae, 3, "N3", {"lambda": lam, "window_days": int(window)})
183     )
184     candidates.sort(key=lambda row: (row[0], row[2], row[1]))
185     best = candidates[0]
186     # 差异不足1%时按预先规则选更简单的N1。
187     near = [row for row in candidates if row[0] <= best[0] * 1.01]
188     chosen = min(near, key=lambda row: (row[2], row[0], row[1]))
189     return ForecastChoice(chosen[3], chosen[4], chosen[0], chosen[1], len(val_days))
190
191
192 def forecast_from_choice(
193     choice: ForecastChoice,
194     net: np.ndarray,
195     load: np.ndarray,
196     pv: np.ndarray,
197     day: int,
198     history_end: int,
199     fallback: np.ndarray,
200     *,
201     lag1_override: np.ndarray | None = None,
202 ) -> np.ndarray:
203     """按已在月首冻结的模型配置生成预测。"""
204
205     if choice.method == "N1":
206         return seasonal_forecast(
207             net,
208             day,
209             float(choice.params["alpha"]),
210             fallback,
211             recursive_lag1=lag1_override,
212         )
213     model = fit_ridge(
214         net,
215         load,
216         pv,
217         history_end,
218         float(choice.params["lambda"]),

```

```

219         int(choice.params["window_days"]),
220         fallback,
221     )
222     return ridge_forecast(
223         model, net, load, pv, day, fallback, lag1_override=lag1_override
224     )
225
226
227 def choose_scenario_count(residuals: np.ndarray) -> int:
228     """按残差分布统计量的0.5%收敛准则从10/20/30中选较小场景数。"""
229
230     n = len(residuals)
231     if n <= 10:
232         return max(1, n)
233     previous = None
234     for size in (10, 20, 30):
235         use = min(size, n)
236         idx = np.linspace(0, n - 1, use).round().astype(int)
237         ordered = residuals[np.argsort(residuals.sum(axis=1))]
238         sample = ordered[idx]
239         signature = np.array(
240             [np.mean(np.maximum(sample, 0.0)), np.quantile(sample, 0.95)]
241         )
242         if previous is not None:
243             denom = np.maximum(np.abs(signature), 1e-9)
244             if float(np.max(np.abs(signature - previous) / denom)) < 0.005:
245                 return max(10, use - 10)
246         previous = signature
247         if use == n:
248             return use
249     return min(30, n)
250
251
252 def residual_scenarios(
253     point: np.ndarray,
254     residuals: np.ndarray,
255     *,
256     scenario_count: int | None = None,
257 ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
258     """用整日残差向量作等概率分位抽样，保留144维日内相关性。"""
259
260     point = np.asarray(point, dtype=float)
261     if residuals.size == 0:

```

```

262         return point[None, :], np.ones(1), np.array([], dtype=int)
263     residuals = np.asarray(residuals, dtype=float)
264     count = scenario_count or choose_scenario_count(residuals)
265     count = min(max(1, int(count)), len(residuals))
266     order = np.argsort(residuals.sum(axis=1))
267     idx_sorted = np.linspace(0, len(residuals) - 1, count).round().astype(int)
268     selected = order[idx_sorted]
269     scenarios = point[None, :] + residuals[selected]
270     probs = np.full(count, 1.0 / count)
271     return scenarios, probs, selected

```

A.5 q2_model.py

```

1  """问题二：严格0时两阶段随机MILP全年滚动求解。
2
3  第一阶段：计划购电G、充电C、放电D、SOC E与互斥状态z跨情景共享。
4  第二阶段：每个净负荷情景只允许紧急购电R和弃电W。
5  输入：附件1固定电价与附件2全年实际负荷/光伏。
6  输出：state/agent_outputs/q2_results.json、results/q2_detail.npz。
7  运行：python code/q2_model.py [--days N] [--horizon-days 1|2]
8  """
9
10 from __future__ import annotations
11
12 import argparse
13 import hashlib
14 import json
15 import platform
16 from dataclasses import asdict, dataclass
17 from datetime import datetime, timezone
18 from pathlib import Path
19 from time import perf_counter
20
21 import numpy as np
22 import scipy
23 from scipy.optimize import Bounds, LinearConstraint, milp
24 from scipy.sparse import lil_matrix
25
26 from causal_forecasts import (
27     ForecastChoice,
28     choose_scenario_count,
29     forecast_from_choice,
30     residual_scenarios,

```

```

31     select_month_model,
32 )
33 from formal_data_loader import DATA_DIR, N_STEPS, load_q2_data, load_typical_day
34
35
36 ROOT = Path(__file__).resolve().parents[1]
37 RESULTS_DIR = ROOT / "results"
38 STATE_DIR = ROOT / "state" / "agent_outputs"
39 ETA_C = 0.9
40 ETA_D = 0.9
41 SOC_MIN = 1200.0
42 SOC_MAX = 10800.0
43 POWER_LIMIT_KWH = 5000.0 / 6.0
44 INITIAL_SOC = 6000.0
45 EMERGENCY_MULTIPLIER = 5.0
46
47
48 @dataclass
49 class StochasticDispatch:
50     grid: np.ndarray
51     charge: np.ndarray
52     discharge: np.ndarray
53     soc: np.ndarray
54     mode: np.ndarray
55     scenario_emergency: np.ndarray
56     scenario_curtailment: np.ndarray
57     objective: float
58     solve_seconds: float
59     success: bool
60     message: str
61     mip_gap: float | None
62     max_balance_residual: float
63     max_soc_residual: float
64
65
66 def solve_two_stage_day(
67     scenario_net_kwh: np.ndarray,
68     probabilities: np.ndarray,
69     price: np.ndarray,
70     initial_soc: float,
71     *,
72     terminal_soc: float | None = None,
73     time_limit: float = 60.0,

```

```

74 ) -> StochasticDispatch:
75     """求解共享计划、仅R/W追索的两阶段随机MILP。"""
76
77     scenarios = np.asarray(scenario_net_kwh, dtype=float)
78     probs = np.asarray(probabilities, dtype=float)
79     tariff = np.asarray(price, dtype=float)
80     if scenarios.ndim != 2:
81         raise ValueError("scenario_net_kwh必须为S×H矩阵")
82     s_count, horizon = scenarios.shape
83     if tariff.shape != (horizon,) or probs.shape != (s_count,):
84         raise ValueError("电价、概率与情景维度不匹配")
85     if np.any(probs < 0) or not np.isclose(probs.sum(), 1.0):
86         raise ValueError("情景概率必须非负且和为1")
87     if np.any(tariff < 0) or not np.isfinite(scenarios).all():
88         raise ValueError("电价不得为负，净负荷情景必须有限")
89
90     # g,c,d,E,z,R,W
91     g0 = 0
92     c0 = g0 + horizon
93     d0 = c0 + horizon
94     e0 = d0 + horizon
95     z0 = e0 + horizon + 1
96     r0 = z0 + horizon
97     w0 = r0 + s_count * horizon
98     nvar = w0 + s_count * horizon
99
100     objective = np.zeros(nvar)
101     objective[g0 : g0 + horizon] = tariff
102     for s in range(s_count):
103         lo = r0 + s * horizon
104         objective[lo : lo + horizon] = probs[s] * EMERGENCY_MULTIPLIER * tariff
105
106     lower = np.zeros(nvar)
107     upper = np.full(nvar, np.inf)
108     upper[c0 : c0 + horizon] = POWER_LIMIT_KWH
109     upper[d0 : d0 + horizon] = POWER_LIMIT_KWH
110     lower[e0 : e0 + horizon + 1] = SOC_MIN
111     upper[e0 : e0 + horizon + 1] = SOC_MAX
112     lower[e0] = upper[e0] = float(initial_soc)
113     if terminal_soc is not None:
114         lower[e0 + horizon] = upper[e0 + horizon] = float(terminal_soc)
115     upper[z0 : z0 + horizon] = 1.0
116

```

```

117 # 每个情景每时段平衡 + 共享SOC状态转移。
118 aeq = lil_matrix((s_count * horizon + horizon, nvar), dtype=float)
119 beq = np.zeros(s_count * horizon + horizon)
120 for s in range(s_count):
121     for t in range(horizon):
122         row = s * horizon + t
123         aeq[row, g0 + t] = 1.0
124         aeq[row, c0 + t] = -1.0
125         aeq[row, d0 + t] = 1.0
126         aeq[row, r0 + s * horizon + t] = 1.0
127         aeq[row, w0 + s * horizon + t] = -1.0
128         beq[row] = scenarios[s, t]
129 for t in range(horizon):
130     row = s_count * horizon + t
131     aeq[row, e0 + t] = -1.0
132     aeq[row, e0 + t + 1] = 1.0
133     aeq[row, c0 + t] = -ETA_C
134     aeq[row, d0 + t] = 1.0 / ETA_D
135
136 # 充放电互斥:  $C \leq Mz$ ,  $D \leq M(1-z)$ 。
137 aub = lil_matrix((2 * horizon, nvar), dtype=float)
138 bub = np.zeros(2 * horizon)
139 for t in range(horizon):
140     aub[t, c0 + t] = 1.0
141     aub[t, z0 + t] = -POWER_LIMIT_KWH
142     aub[horizon + t, d0 + t] = 1.0
143     aub[horizon + t, z0 + t] = POWER_LIMIT_KWH
144     bub[horizon + t] = POWER_LIMIT_KWH
145
146 integrality = np.zeros(nvar, dtype=int)
147 integrality[z0 : z0 + horizon] = 1
148 started = perf_counter()
149 raw = milp(
150     objective,
151     integrality=integrality,
152     bounds=Bounds(lower, upper),
153     constraints=[
154         LinearConstraint(aeq.tocsr(), beq, beq),
155         LinearConstraint(aub.tocsr(), -np.inf, bub),
156     ],
157     options={"time_limit": time_limit, "mip_rel_gap": 1e-7},
158 )
159 elapsed = perf_counter() - started

```

```

160     if raw.x is None:
161         raise RuntimeError(f"Q2两阶段MILP无可用解: {raw.message}")
162     x = raw.x
163     grid = x[g0 : g0 + horizon]
164     charge = x[c0 : c0 + horizon]
165     discharge = x[d0 : d0 + horizon]
166     soc = x[e0 : e0 + horizon + 1]
167     mode = x[z0 : z0 + horizon]
168     emergency = x[r0 : r0 + s_count * horizon].reshape(s_count, horizon)
169     curtailment = x[w0 : w0 + s_count * horizon].reshape(s_count, horizon)
170     balance = (
171         grid[None, :]
172         - charge[None, :]
173         + discharge[None, :]
174         + emergency
175         - curtailment
176         - scenarios
177     )
178     soc_residual = soc[1:] - soc[:-1] - ETA_C * charge + discharge / ETA_D
179     return StochasticDispatch(
180         grid=grid,
181         charge=charge,
182         discharge=discharge,
183         soc=soc,
184         mode=mode,
185         scenario_emergency=emergency,
186         scenario_curtailment=curtailment,
187         objective=float(raw.fun),
188         solve_seconds=elapsed,
189         success=bool(raw.success),
190         message=str(raw.message),
191         mip_gap=float(raw.mip_gap) if getattr(raw, "mip_gap", None) is not None else None,
192         max_balance_residual=float(np.max(np.abs(balance))),
193         max_soc_residual=float(np.max(np.abs(soc_residual))),
194     )
195
196
197 def _sha256(path: Path) -> str:
198     h = hashlib.sha256()
199     with path.open("rb") as handle:
200         for chunk in iter(lambda: handle.read(1024 * 1024), b""):
201             h.update(chunk)
202     return "sha256:" + h.hexdigest()

```

```

203
204
205 def _jsonable_choice(choice: ForecastChoice) -> dict:
206     result = asdict(choice)
207     result["validation_wape"] = (
208         None if not np.isfinite(choice.validation_wape) else choice.validation_wape
209     )
210     result["validation_mae"] = (
211         None if not np.isfinite(choice.validation_mae) else choice.validation_mae
212     )
213     return result
214
215
216 def run_q2(*, days: int = 365, horizon_days: int = 2) -> dict:
217     """从1月1日顺序递推，正式汇总2月1日至12月31日。"""
218
219     if not 1 <= days <= 365 or horizon_days not in (1, 2):
220         raise ValueError("days 须为1..365, horizon_days 须为1或2")
221     typical = load_typical_day()
222     data = load_q2_data()
223     net = data["net_kwh"]
224     load = data["load_kwh"]
225     pv = data["pv_kwh"]
226     dates = data["dates"]
227     price_day = typical["price"]
228
229     shape = (days, N_STEPS)
230     grid = np.zeros(shape)
231     charge = np.zeros(shape)
232     discharge = np.zeros(shape)
233     emergency = np.zeros(shape)
234     curtailment = np.zeros(shape)
235     point_forecast = np.zeros(shape)
236     soc_start = np.zeros(days)
237     soc_end = np.zeros(days)
238     planned_cost = np.zeros(days)
239     emergency_cost = np.zeros(days)
240     solve_seconds = np.zeros(days)
241     scenario_counts = np.zeros(days, dtype=int)
242     max_balance_residual = 0.0
243     max_soc_residual = 0.0
244     simultaneous_count = 0
245     current_soc = INITIAL_SOC

```

```

246     current_choice = ForecastChoice("N1", {"alpha": 1.0}, float("nan"), float("nan"), 0)
247     selection_log: list[dict] = []
248     residual_library: list[np.ndarray] = []
249     day_messages: list[str] = []
250     started_all = perf_counter()
251
252     for day in range(days):
253         date = dates[day]
254         month = int(str(date)[5:7])
255         day_of_month = int(str(date)[8:10])
256         if day >= 31 and day_of_month == 1:
257             current_choice = select_month_model(net, load, pv, day, typical["net_kwh"])
258             selection_log.append(
259                 {
260                     "effective_month": f"2025-{month:02d}",
261                     "training_cutoff": str(dates[day - 1]),
262                     **_jsonable_choice(current_choice),
263                 }
264             )
265
266     # 正式规格：1月1日只使用附件1典型日作为赛前初始化先验。
267     if day == 0:
268         point1 = typical["net_kwh"].copy()
269     else:
270         point1 = forecast_from_choice(
271             current_choice,
272             net,
273             load,
274             pv,
275             day,
276             day,
277             typical["net_kwh"],
278         )
279     point_forecast[day] = point1
280     residuals = (
281         np.vstack(residual_library)
282         if residual_library
283         else np.empty((0, N_STEPS), dtype=float)
284     )
285     count = choose_scenario_count(residuals) if len(residuals) else 1
286     scenarios1, probs, selected = residual_scenarios(
287         point1, residuals, scenario_count=count
288     )

```

```

289
290 use_two_days = horizon_days == 2 and day + 1 < days
291 if use_two_days:
292     point2 = forecast_from_choice(
293         current_choice,
294         net,
295         load,
296         pv,
297         day + 1,
298         day,
299         typical["net_kwh"],
300         lag1_override=point1,
301     )
302     if len(residuals):
303         scenarios2 = point2[None, :] + residuals[selected]
304     else:
305         scenarios2 = point2[None, :]
306     scenarios = np.hstack([scenarios1, scenarios2])
307     tariff = np.tile(price_day, 2)
308 else:
309     scenarios = scenarios1
310     tariff = price_day
311 # 两日滚动窗末端回到当前SOC，消除截断处无代价放空；只执行首日，
312 # 因而不是强迫每日首尾相等。年末单日窗采用同一条件。
313 dispatch = solve_two_stage_day(
314     scenarios, probs, tariff, current_soc, terminal_soc=current_soc
315 )
316
317 plan_g = dispatch.grid[:N_STEPS]
318 plan_c = dispatch.charge[:N_STEPS]
319 plan_d = dispatch.discharge[:N_STEPS]
320 plan_soc = dispatch.soc[: N_STEPS + 1]
321 actual_gap = net[day] + plan_c - plan_g - plan_d
322 actual_r = np.maximum(actual_gap, 0.0)
323 actual_w = np.maximum(-actual_gap, 0.0)
324
325 soc_start[day] = current_soc
326 current_soc = float(plan_soc[-1])
327 soc_end[day] = current_soc
328 grid[day], charge[day], discharge[day] = plan_g, plan_c, plan_d
329 emergency[day], curtailment[day] = actual_r, actual_w
330 planned_cost[day] = float(np.dot(price_day, plan_g))
331 emergency_cost[day] = float(np.dot(EMERGENCY_MULTIPLIER * price_day, actual_r))

```

```

332     solve_seconds[day] = dispatch.solve_seconds
333     scenario_counts[day] = scenarios1.shape[0]
334     max_balance_residual = max(max_balance_residual, dispatch.max_balance_residual)
335     max_soc_residual = max(max_soc_residual, dispatch.max_soc_residual)
336     simultaneous_count += int(np.sum((plan_c > 1e-7) & (plan_d > 1e-7)))
337     day_messages.append(dispatch.message)
338
339     # 当天结束后才允许实际误差进入次日残差库。
340     residual_library.append(net[day] - point1)
341
342     elapsed_all = perf_counter() - started_all
343     official_start = 31 if days > 31 else days
344     sl = slice(official_start, days)
345     actual_total = planned_cost + emergency_cost
346     forecast_error = point_forecast - net[:days]
347     wape = float(
348         np.sum(np.abs(forecast_error[sl])) / np.sum(np.abs(net[:days][sl]))
349     ) if official_start < days else None
350     mae = float(np.mean(np.abs(forecast_error[sl]))) if official_start < days else None
351     rmse = float(np.sqrt(np.mean(forecast_error[sl] ** 2))) if official_start < days else None
352     bias = float(np.mean(forecast_error[sl])) if official_start < days else None
353
354     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
355     STATE_DIR.mkdir(parents=True, exist_ok=True)
356     np.savez_compressed(
357         RESULTS_DIR / "q2_detail.npz",
358         dates=dates[:days],
359         price=price_day,
360         actual_net_kwh=net[:days],
361         point_forecast_net_kwh=point_forecast,
362         planned_grid_kwh=grid,
363         planned_charge_kwh=charge,
364         planned_discharge_kwh=discharge,
365         actual_emergency_kwh=emergency,
366         actual_curtailment_kwh=curtailment,
367         soc_start_kwh=soc_start,
368         soc_end_kwh=soc_end,
369         planned_cost_yuan=planned_cost,
370         emergency_cost_yuan=emergency_cost,
371         actual_total_cost_yuan=actual_total,
372         scenario_count=scenario_counts,
373         official_start_index=np.array([official_start]),
374     )

```

```

375
376 official_days = max(0, days - official_start)
377 output = {
378     "sub_question": "Q2",
379     "status": "success" if (
380         max_balance_residual <= 1e-5
381         and max_soc_residual <= 1e-5
382         and simultaneous_count == 0
383     ) else "failed",
384     "model": "严格0时两阶段随机MILP（计划G/C/D/E/z共享，情景仅R/W）",
385     "run_timestamp": datetime.now(timezone.utc).astimezone().isoformat(),
386     "input_data_hash": _sha256(DATA_DIR / "附件2.xlsx"),
387     "information_boundary": {
388         "forecast_origin": "每天00:00",
389         "history_rule": "日期d的预测、参数和残差样本仅使用j<d的已完成数据",
390         "january_warm_start": "2025-01-01使用附件1典型日先验；其后逐日因果更新SOC与残差库",
391         "first_stage_shared": ["G", "C", "D", "E", "z"],
392         "scenario_recourse_only": ["R", "W"],
393     },
394     "key_results": {
395         "evaluation_period": (
396             "2025-02-01至" + str(dates[days - 1]) if official_days else "尚未进入正式评价期"
397         ),
398         "evaluation_days": official_days,
399         "planned_purchase_kwh": float(grid[s1].sum()),
400         "planned_charge_kwh": float(charge[s1].sum()),
401         "planned_discharge_kwh": float(discharge[s1].sum()),
402         "emergency_purchase_kwh": float(emergency[s1].sum()),
403         "curtailment_kwh": float(curtailment[s1].sum()),
404         "planned_cost_yuan": float(planned_cost[s1].sum()),
405         "emergency_cost_yuan": float(emergency_cost[s1].sum()),
406         "actual_total_cost_yuan": float(actual_total[s1].sum()),
407         "emergency_intervals": int(np.sum(emergency[s1] > 1e-7)),
408         "minimum_soc_kwh": float(min(soc_start[s1].min(initial=INITIAL_SOC), soc_end[s1].
min(initial=INITIAL_SOC))),
409         "maximum_soc_kwh": float(max(soc_start[s1].max(initial=INITIAL_SOC), soc_end[s1].
max(initial=INITIAL_SOC))),
410     },
411     "forecast_metrics": {"WAPE": wape, "MAE_kwh": mae, "RMSE_kwh": rmse, "Bias_kwh": bias},
412     "parameter_selection": {
413         "monthly_log": selection_log,
414         "scenario_count_rule": "整日残差按总误差排序作等概率分位抽样；10/20/30按统计签名0.5%收敛取较小者",

```

```

415         "scenario_count_min": int(scenario_counts.min()),
416         "scenario_count_max": int(scenario_counts.max()),
417         "randomness": "无; 分位抽样完全确定性复现",
418     },
419     "intermediate_results": {
420         "constraint_check": "all_passed" if (
421             max_balance_residual <= 1e-5
422             and max_soc_residual <= 1e-5
423             and simultaneous_count == 0
424             and np.all(soc_start >= SOC_MIN - 1e-6)
425             and np.all(soc_end <= SOC_MAX + 1e-6)
426         ) else "failed",
427         "max_scenario_balance_residual_kwh": max_balance_residual,
428         "max_soc_residual_kwh": max_soc_residual,
429         "simultaneous_charge_discharge_intervals": simultaneous_count,
430         "soc_link_max_residual_kwh": float(np.max(np.abs(soc_start[1:] - soc_end[:-1]))) if
days > 1 else 0.0,
431         "all_solver_runs_have_solution": len(day_messages) == days,
432         "total_runtime_seconds": elapsed_all,
433         "mean_daily_solve_seconds": float(solve_seconds.mean()),
434         "max_daily_solve_seconds": float(solve_seconds.max()),
435     },
436     "environment": {
437         "python": platform.python_version(),
438         "numpy": np.__version__,
439         "scipy": scipy.__version__,
440         "solver": "scipy.optimize.milp (HiGHS)",
441     },
442     "artifacts": {"detail": "results/q2_detail.npz", "code": "code/q2_model.py"},
443     "warnings": [
444         "两日滚动视野的第二日只作为终端延续近似, 执行时仅冻结首日计划; 视野末端SOC回到视野初值
以抑制截断放空。"
445     ] if horizon_days == 2 else [
446         "单日视野可能低估日末储能的跨日价值。"
447     ],
448 }
449 with (STATE_DIR / "q2_results.json").open("w", encoding="utf-8") as handle:
450     json.dump(output, handle, ensure_ascii=False, indent=2)
451 return output
452
453
454 def main() -> None:
455     parser = argparse.ArgumentParser()

```

```

456     parser.add_argument("--days", type=int, default=365)
457     parser.add_argument("--horizon-days", type=int, choices=(1, 2), default=2)
458     args = parser.parse_args()
459     result = run_q2(days=args.days, horizon_days=args.horizon_days)
460     print(json.dumps({
461         "key_results": result["key_results"],
462         "forecast_metrics": result["forecast_metrics"],
463         "checks": result["intermediate_results"],
464     }, ensure_ascii=False, indent=2))
465
466
467 if __name__ == "__main__":
468     main()

```

A.6 q34_helpers.py

```

1  """Q3/Q4 共用的因果预测、滚动调整 MILP 与回测工具。
2
3  所有功率输入在载入后乘 1/6，内部决策量均为每十分钟电量（kWh）。
4  附件中的未来真实负荷、光伏和价格只进入事后结算，不进入预测函数。
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass
10 from pathlib import Path
11 from time import perf_counter
12 from typing import Callable, Iterable
13
14 import numpy as np
15 import pandas as pd
16 from scipy.optimize import Bounds, LinearConstraint, milp
17 from scipy.sparse import lil_matrix
18
19 from common_milp import solve_deterministic_dispatch
20
21
22 STEP_HOURS = 1.0 / 6.0
23 SLOTS_PER_DAY = 144
24 ORIGIN_SLOTS = (0, 36, 72, 108)
25 SOC_MIN = 1200.0
26 SOC_MAX = 10800.0
27 INTERVAL_LIMIT = 5000.0 * STEP_HOURS

```

```

28 ETA_C = 0.9
29 ETA_D = 0.9
30
31
32 @dataclass
33 class CData:
34     dates: pd.DatetimeIndex
35     load: np.ndarray
36     pv_actual: np.ndarray
37     fixed_price: np.ndarray
38     realtime_price: np.ndarray
39     typical_load: np.ndarray
40     typical_pv: np.ndarray
41     pv_hourly_forecast: np.ndarray
42
43
44 def load_c_data(root: Path) -> CData:
45     """按工作表位置读取附件，避免依赖中文列名编码。"""
46     a1 = pd.read_excel(root / "附件1.xlsx")
47     typical_load = a1.iloc[:, 2].to_numpy(float) * STEP_HOURS
48     typical_pv = a1.iloc[:, 3].to_numpy(float) * STEP_HOURS
49     fixed_price = a1.iloc[:, 1].to_numpy(float)
50
51     load_df = pd.read_excel(root / "附件2.xlsx", sheet_name=0)
52     pv_df = pd.read_excel(root / "附件2.xlsx", sheet_name=1)
53     dates = pd.DatetimeIndex(pd.to_datetime(load_df.iloc[:, 0]))
54     load = load_df.iloc[:, 1:].to_numpy(float) * STEP_HOURS
55     pv_actual = pv_df.iloc[:, 1:].to_numpy(float) * STEP_HOURS
56
57     price_df = pd.read_excel(root / "附件4.xlsx")
58     realtime_price = price_df.iloc[:, 1:].to_numpy(float)
59
60     f = pd.read_excel(root / "附件3.xlsx")
61     f.iloc[:, 0] = f.iloc[:, 0].ffill()
62     f_dates = pd.to_datetime(f.iloc[:, 0])
63     f_origins = f.iloc[:, 1].astype(str).str.extract(r"(\d+)", expand=False).astype(int)
64     forecasts = f.iloc[:, 2:26].to_numpy(float) * STEP_HOURS
65     pv_hourly = np.full((len(dates), 4, 24), np.nan)
66     date_pos = {d.normalize(): i for i, d in enumerate(dates)}
67     origin_pos = {0: 0, 6: 1, 12: 2, 18: 3}
68     for i, d in enumerate(f_dates):
69         di = date_pos[d.normalize()]
70         pv_hourly[di, origin_pos[int(f_origins.iloc[i])], :] = forecasts[i]

```

```

71     if np.isnan(pv_hourly).any():
72         raise ValueError("附件3光伏预测存在未映射缺失值")
73     return CData(
74         dates=dates,
75         load=load,
76         pv_actual=pv_actual,
77         fixed_price=fixed_price,
78         realtime_price=realtime_price,
79         typical_load=typical_load,
80         typical_pv=typical_pv,
81         pv_hourly_forecast=pv_hourly,
82     )
83
84
85 def pv_forecast_10min(data: CData, day: int, origin: int) -> np.ndarray:
86     """仅用本次发布的24个整点预测和发布时刻已观测锚点插值。"""
87     oi = ORIGIN_SLOTS.index(origin)
88     hourly = data.pv_hourly_forecast[day, oi]
89     if origin == 0:
90         anchor = data.pv_actual[day - 1, -1] if day > 0 else data.typical_pv[-1]
91     else:
92         anchor = data.pv_actual[day, origin - 1]
93     horizon = SLOTS_PER_DAY - origin
94     x = np.arange(25, dtype=float)
95     y = np.r_[anchor, hourly]
96     target = np.arange(1, horizon + 1, dtype=float) / 6.0
97     return np.maximum(np.interp(target, x, y), 0.0)
98
99
100 def _seasonal_curve(values: np.ndarray, typical: np.ndarray, day: int, alpha: float) -> np.
    ndarray:
101     if day <= 0:
102         return typical.copy()
103     lag1 = values[day - 1]
104     lag7 = values[day - 7] if day >= 7 else typical
105     return alpha * lag1 + (1.0 - alpha) * lag7
106
107
108 def predict_load(
109     data: CData,
110     day: int,
111     origin: int,
112     params: tuple[float, int, float],

```

```

113 ) -> np.ndarray:
114     """日前季节基线加仅由已实现前缀计算的稳健偏差校正。"""
115     alpha, window, rho = params
116     baseline = _seasonal_curve(data.load, data.typical_load, day, alpha)
117     future = baseline[origin:].copy()
118     if origin > 0:
119         lo = max(0, origin - window)
120         correction = float(np.median(data.load[day, lo:origin] - baseline[lo:origin]))
121         lead = np.arange(1, len(future) + 1, dtype=float)
122         future += np.power(rho, lead / 36.0) * correction
123     return np.maximum(future, 0.0)
124
125
126 LOAD_PARAM_GRID = tuple(
127     (a, w, r)
128     for a in (0.0, 0.25, 0.5, 0.75, 1.0)
129     for w in (6, 18, 36)
130     for r in (0.25, 0.5, 0.75, 1.0)
131 )
132
133
134 def select_load_params(data: CData, day: int, lookback: int = 14) -> tuple[float, int, float]:
135     """在预测日之前最近若干完整日上，以真正执行的首个6小时 WAPE 选参。"""
136     if day < 2:
137         return (0.5, 18, 0.75)
138     val_days = range(max(1, day - lookback), day)
139     best = None
140     for params in LOAD_PARAM_GRID:
141         ae = 0.0
142         denom = 0.0
143         for vd in val_days:
144             for origin in ORIGIN_SLOTS:
145                 pred = predict_load(data, vd, origin, params)[:36]
146                 actual = data.load[vd, origin : origin + len(pred)]
147                 ae += float(np.abs(pred - actual).sum())
148                 denom += float(np.abs(actual).sum())
149             score = ae / max(denom, 1e-9)
150             if best is None or score < best[0] - 1e-12:
151                 best = (score, params)
152     return best[1]
153
154
155 def predict_price_p1(

```

```

156     data: CData, day: int, origin: int, alpha: float
157 ) -> np.ndarray:
158     baseline = _seasonal_curve(data.realtime_price, data.fixed_price, day, alpha)
159     return np.maximum(baseline[origin:], 1e-4)
160
161
162 def predict_price_p2(
163     data: CData,
164     day: int,
165     origin: int,
166     params: tuple[int, int, float],
167 ) -> np.ndarray:
168     k, window, rho = params
169     same_weekday = [day - 7 * j for j in range(1, k + 1) if day - 7 * j >= 0]
170     if same_weekday:
171         baseline = np.median(data.realtime_price[same_weekday], axis=0)
172     elif day > 0:
173         baseline = data.realtime_price[day - 1].copy()
174     else:
175         baseline = data.fixed_price.copy()
176     future = baseline[origin:].copy()
177     if origin > 0:
178         lo = max(0, origin - window)
179         correction = float(
180             np.median(data.realtime_price[day, lo:origin] - baseline[lo:origin])
181         )
182         lead = np.arange(1, len(future) + 1, dtype=float)
183         future += np.power(rho, lead / 36.0) * correction
184     hist_max = (
185         float(np.max(data.realtime_price[:day]))
186         if day > 0
187         else float(np.max(data.fixed_price))
188     )
189     return np.clip(future, 1e-4, hist_max)
190
191
192 PRICE_P1_GRID = tuple(("P1", a) for a in (0.0, 0.25, 0.5, 0.75, 1.0))
193 PRICE_P2_GRID = tuple(
194     ("P2", (k, w, r))
195     for k in (2, 4, 8)
196     for w in (6, 18, 36)
197     for r in (0.25, 0.5, 0.75, 1.0)
198 )

```

```

199
200
201 def select_price_params(data: CData, day: int, origin: int, lookback: int = 14):
202     """仅在当前预测原点以前的完整日上比较 P1/P2 的 WAPE。"""
203     if day < 2:
204         return ("P1", 0.5)
205     val_days = range(max(1, day - lookback), day)
206     best = None
207     for model, params in PRICE_P1_GRID + PRICE_P2_GRID:
208         ae = 0.0
209         denom = 0.0
210         for vd in val_days:
211             pred = (
212                 predict_price_p1(data, vd, origin, params)
213                 if model == "P1"
214                 else predict_price_p2(data, vd, origin, params)
215             )
216             actual = data.realtime_price[vd, origin:]
217             ae += float(np.abs(pred - actual).sum())
218             denom += float(np.abs(actual).sum())
219             score = ae / max(denom, 1e-9)
220             if best is None or score < best[0] - 1e-12:
221                 best = (score, model, params)
222     return (best[1], best[2])
223
224
225 def predict_price(data: CData, day: int, origin: int, selected) -> np.ndarray:
226     model, params = selected
227     if model == "P1":
228         return predict_price_p1(data, day, origin, params)
229     return predict_price_p2(data, day, origin, params)
230
231
232 def solve_adjustment_dispatch(
233     load_kwh: np.ndarray,
234     pv_kwh: np.ndarray,
235     price: np.ndarray,
236     initial_soc: float,
237     old_grid: np.ndarray,
238     *,
239     terminal_soc: float | None = None,
240     binary_mode: bool = True,
241     time_limit: float = 120.0,

```

```

242 ) -> dict[str, np.ndarray | float | bool | str]:
243     """最小化未来增减购结算与预测紧急购电费。"""
244     load = np.asarray(load_kwh, float)
245     pv = np.asarray(pv_kwh, float)
246     tariff = np.asarray(price, float)
247     old = np.asarray(old_grid, float)
248     n = len(load)
249     if not (len(pv) == len(tariff) == len(old) == n):
250         raise ValueError("调整模型各输入长度不一致")
251
252     # g,c,d,w,r,E,z,inc,dec
253     g = slice(0, n)
254     c = slice(n, 2 * n)
255     d = slice(2 * n, 3 * n)
256     w = slice(3 * n, 4 * n)
257     emergency = slice(4 * n, 5 * n)
258     e = slice(5 * n, 6 * n + 1)
259     z = slice(6 * n + 1, 7 * n + 1)
260     inc = slice(7 * n + 1, 8 * n + 1)
261     dec = slice(8 * n + 1, 9 * n + 1)
262     nvar = 9 * n + 1
263
264     obj = np.zeros(nvar)
265     obj[emergency] = 5.0 * tariff
266     obj[inc] = 1.5 * tariff
267     obj[dec] = -0.5 * tariff
268     lower = np.zeros(nvar)
269     upper = np.full(nvar, np.inf)
270     upper[c] = INTERVAL_LIMIT
271     upper[d] = INTERVAL_LIMIT
272     upper[w] = pv
273     lower[e], upper[e] = SOC_MIN, SOC_MAX
274     lower[e.start] = upper[e.start] = initial_soc
275     if terminal_soc is not None:
276         lower[e.stop - 1] = upper[e.stop - 1] = terminal_soc
277     upper[z] = 1.0
278
279     # 能量平衡、SOC、计划差额恒等式。
280     aeq = lil_matrix((3 * n, nvar), dtype=float)
281     beq = np.zeros(3 * n)
282     for t in range(n):
283         aeq[t, g.start + t] = 1.0
284         aeq[t, c.start + t] = -1.0

```

```

285     aeq[t, d.start + t] = 1.0
286     aeq[t, w.start + t] = -1.0
287     aeq[t, emergency.start + t] = 1.0
288     beq[t] = load[t] - pv[t]
289
290     row = n + t
291     aeq[row, e.start + t] = -1.0
292     aeq[row, e.start + t + 1] = 1.0
293     aeq[row, c.start + t] = -ETA_C
294     aeq[row, d.start + t] = 1.0 / ETA_D
295
296     row = 2 * n + t
297     aeq[row, g.start + t] = 1.0
298     aeq[row, inc.start + t] = -1.0
299     aeq[row, dec.start + t] = 1.0
300     beq[row] = old[t]
301
302     aub = lil_matrix((2 * n, nvar), dtype=float)
303     bub = np.zeros(2 * n)
304     for t in range(n):
305         aub[t, c.start + t] = 1.0
306         aub[t, z.start + t] = -INTERVAL_LIMIT
307         aub[n + t, d.start + t] = 1.0
308         aub[n + t, z.start + t] = INTERVAL_LIMIT
309         bub[n + t] = INTERVAL_LIMIT
310
311     integrality = np.zeros(nvar, dtype=int)
312     if binary_mode:
313         integrality[z] = 1
314     started = perf_counter()
315     raw = milp(
316         obj,
317         integrality=integrality,
318         bounds=Bounds(lower, upper),
319         constraints=[
320             LinearConstraint(aeq.tocsr(), beq, beq),
321             LinearConstraint(aub.tocsr(), -np.inf, bub),
322         ],
323         options={"time_limit": time_limit, "mip_rel_gap": 1e-8},
324     )
325     elapsed = perf_counter() - started
326     if raw.x is None:
327         raise RuntimeError(f"调整MILP求解失败: {raw.message}")

```

```

328     x = raw.x
329     balance = x[g] + pv + x[d] + x[emergency] - load - x[c] - x[w]
330     soc_res = x[e][1:] - x[e][: -1] - ETA_C * x[c] + x[d] / ETA_D
331     return {
332         "grid": x[g],
333         "charge": x[c],
334         "discharge": x[d],
335         "curtailment": x[w],
336         "predicted_emergency": x[emergency],
337         "soc": x[e],
338         "increase": x[inc],
339         "decrease": x[dec],
340         "objective": float(raw.fun),
341         "solve_seconds": elapsed,
342         "success": bool(raw.success),
343         "message": str(raw.message),
344         "max_balance_residual": float(np.max(np.abs(balance))),
345         "max_soc_residual": float(np.max(np.abs(soc_res))),
346     }
347
348
349 def _initial_dispatch(load, pv, price, initial_soc, terminal_soc=None):
350     raw = solve_deterministic_dispatch(
351         load,
352         pv,
353         price,
354         initial_soc,
355         terminal_soc=terminal_soc,
356         allow_emergency=True,
357         emergency_multiplier=5.0,
358         binary_mode=True,
359     )
360     return {
361         "grid": raw.grid,
362         "charge": raw.charge,
363         "discharge": raw.discharge,
364         "curtailment": raw.curtailment,
365         "predicted_emergency": raw.emergency,
366         "soc": raw.soc,
367         "increase": np.zeros_like(raw.grid),
368         "decrease": np.zeros_like(raw.grid),
369         "objective": raw.objective,
370         "solve_seconds": raw.solve_seconds,

```

```

371         "success": raw.success,
372         "message": raw.message,
373         "max_balance_residual": raw.max_balance_residual,
374         "max_soc_residual": raw.max_soc_residual,
375     }
376
377
378 def simulate_mpc(
379     data: CData,
380     day_indices: Iterable[int],
381     initial_soc: float,
382     origins: tuple[int, ...],
383     planning_price_provider: Callable[[int, int], np.ndarray],
384     settlement_price: np.ndarray,
385     *,
386     keep_versions: bool = False,
387 ) -> tuple[dict[str, np.ndarray], float, list[dict]]:
388     """逐日、逐预测原点顺序回测，严格冻结已执行时段。"""
389     days = list(day_indices)
390     nday = len(days)
391     shape = (nday, SLOTS_PER_DAY)
392     out = {
393         "initial_grid": np.zeros(shape),
394         "effective_grid": np.zeros(shape),
395         "charge": np.zeros(shape),
396         "discharge": np.zeros(shape),
397         "emergency": np.zeros(shape),
398         "curtailment": np.zeros(shape),
399         "soc": np.zeros((nday, SLOTS_PER_DAY + 1)),
400         "planned_cost": np.zeros(nday),
401         "adjustment_cost": np.zeros(nday),
402         "emergency_cost": np.zeros(nday),
403         "total_cost": np.zeros(nday),
404         "solve_seconds": np.zeros(nday),
405         "max_balance_residual": np.zeros(nday),
406         "max_soc_residual": np.zeros(nday),
407     }
408     versions: list[dict] = []
409     current_soc = float(initial_soc)
410     selected_load_params: dict[int, tuple[float, int, float]] = {}
411     for row, day in enumerate(days):
412         selected_load_params[day] = select_load_params(data, day)
413         g_eff = np.zeros(SLOTS_PER_DAY)

```

```

414 c_eff = np.zeros(SLOTS_PER_DAY)
415 d_eff = np.zeros(SLOTS_PER_DAY)
416 day_soc = np.zeros(SLOTS_PER_DAY + 1)
417 day_soc[0] = current_soc
418 day_terminal_target = current_soc
419 day_versions = []
420 for oi, origin in enumerate(origins):
421     next_origin = origins[oi + 1] if oi + 1 < len(origins) else SLOTS_PER_DAY
422     load_hat = predict_load(data, day, origin, selected_load_params[day])
423     pv_hat = pv_forecast_10min(data, day, origin)
424     price_hat = planning_price_provider(day, origin)
425     if not (len(load_hat) == len(pv_hat) == len(price_hat) == SLOTS_PER_DAY - origin):
426         raise ValueError("预测时域长度不一致")
427     if origin == 0:
428         # 用48小时延伸窗估计跨日续接价值，仅执行前24小时。
429         # 第二天曲线由当时可见的当前预测重复得到，不读取次日实际值；
430         # 延伸窗末端回到窗初SOC，故日末SOC可随调度自然变化。
431         tail_load = load_hat.copy()
432         tail_pv = pv_hat.copy()
433         tail_price = price_hat.copy()
434         solved_long = _initial_dispatch(
435             np.r_[load_hat, tail_load],
436             np.r_[pv_hat, tail_pv],
437             np.r_[price_hat, tail_price],
438             current_soc,
439             terminal_soc=current_soc,
440         )
441         solved = {
442             key: (value[:SLOTS_PER_DAY] if isinstance(value, np.ndarray) and key != "
soc" else value)
443                 for key, value in solved_long.items()
444             }
445         solved["soc"] = solved_long["soc"][: SLOTS_PER_DAY + 1]
446         day_terminal_target = float(solved_long["soc"][SLOTS_PER_DAY])
447         g_eff[:] = solved["grid"]
448         c_eff[:] = solved["charge"]
449         d_eff[:] = solved["discharge"]
450         out["initial_grid"][row] = g_eff
451         old = np.zeros_like(g_eff)
452     else:
453         old = g_eff.copy()
454         solved = solve_adjustment_dispatch(
455             load_hat,

```

```

456         pv_hat,
457         price_hat,
458         day_soc[origin],
459         old[origin:],
460         terminal_soc=day_terminal_target,
461     )
462     g_eff[origin:] = solved["grid"]
463     c_eff[origin:] = solved["charge"]
464     d_eff[origin:] = solved["discharge"]
465     delta_plus = np.maximum(g_eff[origin:] - old[origin:], 0.0)
466     delta_minus = np.maximum(old[origin:] - g_eff[origin:], 0.0)
467     settle = settlement_price[day, origin:]
468     out["adjustment_cost"][row] += float(
469         np.sum(1.5 * settle * delta_plus - 0.5 * settle * delta_minus)
470     )
471     out["solve_seconds"][row] += float(solved["solve_seconds"])
472     out["max_balance_residual"][row] = max(
473         out["max_balance_residual"][row], float(solved["max_balance_residual"])
474     )
475     out["max_soc_residual"][row] = max(
476         out["max_soc_residual"][row], float(solved["max_soc_residual"])
477     )
478     if keep_versions:
479         day_versions.append(
480             {
481                 "origin": origin,
482                 "old_grid": old[origin:].copy(),
483                 "new_grid": g_eff[origin:].copy(),
484                 "load_forecast": load_hat.copy(),
485                 "pv_forecast": pv_hat.copy(),
486                 "price_forecast": price_hat.copy(),
487             }
488         )
489
490     # 只执行到下一次允许更新的时刻；实际偏差仅用紧急购电/弃电平衡。
491     sl = slice(origin, next_origin)
492     supply_without_rec = g_eff[sl] + data.pv_actual[day, sl] + d_eff[sl]
493     demand = data.load[day, sl] + c_eff[sl]
494     emergency = np.maximum(demand - supply_without_rec, 0.0)
495     curtailment = np.maximum(supply_without_rec - demand, 0.0)
496     out["emergency"][row, sl] = emergency
497     out["curtailment"][row, sl] = curtailment
498     for t in range(origin, next_origin):

```

```

499         day_soc[t + 1] = day_soc[t] + ETA_C * c_eff[t] - d_eff[t] / ETA_D
500
501     out["effective_grid"][row] = g_eff
502     out["charge"][row] = c_eff
503     out["discharge"][row] = d_eff
504     out["soc"][row] = day_soc
505     prices = settlement_price[day]
506     out["planned_cost"][row] = float(np.sum(prices * out["initial_grid"][row]))
507     out["emergency_cost"][row] = float(np.sum(5.0 * prices * out["emergency"][row]))
508     out["total_cost"][row] = (
509         out["planned_cost"][row]
510         + out["adjustment_cost"][row]
511         + out["emergency_cost"][row]
512     )
513     actual_balance = (
514         g_eff
515         + data.pv_actual[day]
516         + d_eff
517         + out["emergency"][row]
518         - data.load[day]
519         - c_eff
520         - out["curtailment"][row]
521     )
522     out["max_balance_residual"][row] = max(
523         out["max_balance_residual"][row], float(np.max(np.abs(actual_balance)))
524     )
525     soc_res = day_soc[1:] - day_soc[:-1] - ETA_C * c_eff + d_eff / ETA_D
526     out["max_soc_residual"][row] = max(
527         out["max_soc_residual"][row], float(np.max(np.abs(soc_res)))
528     )
529     if np.min(day_soc) < SOC_MIN - 1e-5 or np.max(day_soc) > SOC_MAX + 1e-5:
530         raise AssertionError("SOC越界")
531     if np.max(np.minimum(c_eff, d_eff)) > 1e-5:
532         raise AssertionError("出现同时充放电")
533     current_soc = float(day_soc[-1])
534     if keep_versions:
535         versions.append({"day": day, "versions": day_versions})
536     return out, current_soc, versions
537
538
539 def summarize_run(arrays: dict[str, np.ndarray], start_row: int = 0) -> dict[str, float]:
540     s = slice(start_row, None)
541     return {

```

```

542         "total_cost_yuan": float(np.sum(arrays["total_cost"][s])),
543         "planned_cost_yuan": float(np.sum(arrays["planned_cost"][s])),
544         "adjustment_cost_yuan": float(np.sum(arrays["adjustment_cost"][s])),
545         "emergency_cost_yuan": float(np.sum(arrays["emergency_cost"][s])),
546         "emergency_energy_kwh": float(np.sum(arrays["emergency"][s])),
547         "curtailment_energy_kwh": float(np.sum(arrays["curtailment"][s])),
548         "grid_energy_kwh": float(np.sum(arrays["effective_grid"][s])),
549         "minimum_soc_kwh": float(np.min(arrays["soc"][s])),
550         "maximum_soc_kwh": float(np.max(arrays["soc"][s])),
551         "solve_seconds": float(np.sum(arrays["solve_seconds"][s])),
552         "max_balance_residual_kwh": float(np.max(arrays["max_balance_residual"][s])),
553         "max_soc_residual_kwh": float(np.max(arrays["max_soc_residual"][s])),
554     }

```

A.7 q3_model.py

```

1  """问题三：0/6/12/18 时四次光伏预测驱动的确定性 MILP-MPC。
2
3  输入：附件2实际负荷/光伏、附件3分时发布光伏预测、附件1固定电价。
4  输出：results/q3_results.json、q3_daily.csv、q3_detail.npz。
5  运行：python code/q3_model.py --full --ablation
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import json
12 from datetime import datetime
13 from pathlib import Path
14
15 import numpy as np
16 import pandas as pd
17
18 from q34_helpers import (
19     ORIGIN_SLOTS,
20     load_c_data,
21     select_load_params,
22     simulate_mpc,
23     summarize_run,
24 )
25
26
27 ROOT = Path(__file__).resolve().parents[1]

```

```

28 ATTACHMENTS = ROOT / "problems" / "选题C_2026正式" / "附件"
29 RESULTS = ROOT / "results"
30 STATE_OUTPUTS = ROOT / "state" / "agent_outputs"
31 SPECIFIED_DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
32
33
34 def _price_provider(data):
35     return lambda day, origin: data.fixed_price[origin:].copy()
36
37
38 def _version_npz(versions, nday: int):
39     grid = np.full((nday, 4, 144), np.nan)
40     load_hat = np.full((nday, 4, 144), np.nan)
41     pv_hat = np.full((nday, 4, 144), np.nan)
42     for row, day_record in enumerate(versions):
43         for ver in day_record["versions"]:
44             oi = ORIGIN_SLOTS.index(ver["origin"])
45             origin = ver["origin"]
46             grid[row, oi, origin:] = ver["new_grid"]
47             load_hat[row, oi, origin:] = ver["load_forecast"]
48             pv_hat[row, oi, origin:] = ver["pv_forecast"]
49     return grid, load_hat, pv_hat
50
51
52 def run(full: bool, ablation: bool) -> dict:
53     RESULTS.mkdir(parents=True, exist_ok=True)
54     data = load_c_data(ATTACHMENTS)
55     nday = len(data.dates) if full else min(14, len(data.dates))
56     days = range(nday)
57     arrays, _, versions = simulate_mpc(
58         data,
59         days,
60         6000.0,
61         ORIGIN_SLOTS,
62         _price_provider(data),
63         np.tile(data.fixed_price, (len(data.dates), 1)),
64         keep_versions=True,
65     )
66     report_start = 31 if nday > 31 else 0
67     main_summary = summarize_run(arrays, report_start)
68
69     comparisons = {"0_6_12_18": main_summary}
70     if ablation:

```

```

71     for name, origins in {
72         "0_only": (0,),
73         "0_6": (0, 36),
74         "0_6_12": (0, 36, 72),
75     }.items():
76         comp, _, _ = simulate_mpc(
77             data,
78             days,
79             6000.0,
80             origins,
81             _price_provider(data),
82             np.tile(data.fixed_price, (len(data.dates), 1)),
83         )
84         comparisons[name] = summarize_run(comp, report_start)
85
86     dates = data.dates[:nday]
87     daily = pd.DataFrame(
88         {
89             "date": dates.strftime("%Y-%m-%d"),
90             "planned_cost_yuan": arrays["planned_cost"],
91             "adjustment_cost_yuan": arrays["adjustment_cost"],
92             "emergency_cost_yuan": arrays["emergency_cost"],
93             "total_cost_yuan": arrays["total_cost"],
94             "emergency_energy_kwh": arrays["emergency"].sum(axis=1),
95             "grid_energy_kwh": arrays["effective_grid"].sum(axis=1),
96             "minimum_soc_kwh": arrays["soc"].min(axis=1),
97             "solve_seconds": arrays["solve_seconds"],
98         }
99     )
100     daily.to_csv(RESULTS / "q3_daily.csv", index=False, encoding="utf-8-sig")
101     grid_versions, load_forecasts, pv_forecasts = _version_npz(versions, nday)
102     np.savez_compressed(
103         RESULTS / "q3_detail.npz",
104         dates=dates.strftime("%Y-%m-%d").to_numpy(),
105         **arrays,
106         grid_versions=grid_versions,
107         load_forecasts=load_forecasts,
108         pv_forecasts=pv_forecasts,
109     )
110
111     selected_days = {}
112     for date in SPECIFIED_DATES:
113         match = np.flatnonzero(dates.strftime("%Y-%m-%d") == date)

```

```

114     if len(match):
115         i = int(match[0])
116         selected_days[date] = {
117             "planned_cost_yuan": float(arrays["planned_cost"][i]),
118             "adjustment_cost_yuan": float(arrays["adjustment_cost"][i]),
119             "emergency_cost_yuan": float(arrays["emergency_cost"][i]),
120             "total_cost_yuan": float(arrays["total_cost"][i]),
121             "emergency_energy_kwh": float(arrays["emergency"][i].sum()),
122             "minimum_soc_kwh": float(arrays["soc"][i].min()),
123             "load_predictor_params": list(select_load_params(data, i)),
124         }
125
126     result = {
127         "sub_question": "Q3",
128         "status": "success" if main_summary["max_balance_residual_kwh"] <= 1e-5 and
129         main_summary["max_soc_residual_kwh"] <= 1e-5 else "failed",
130         "model": "确定性MILP-MPC (0/6/12/18滚动更新)",
131         "run_timestamp": datetime.now().astimezone().isoformat(),
132         "scope": {
133             "simulation_start": str(dates[0].date()),
134             "simulation_end": str(dates[-1].date()),
135             "reported_period": (
136                 f"{dates[report_start].date()}至{dates[-1].date()}" if nday else ""
137             ),
138             "settlement_rule": {
139                 "initial_plan": "p*G0",
140                 "increase": "+1.5*p*max(Gnew-Gold,0)",
141                 "decrease": "-0.5*p*max(Gold-Gnew,0)",
142                 "emergency": "+5*p*Ractual",
143                 "comparison_base": "immediately_previous_effective_plan",
144             },
145             "key_results": main_summary,
146             "update_ablation": comparisons,
147             "specified_days": selected_days,
148             "constraint_check": {
149                 "all_passed": bool(
150                     main_summary["max_balance_residual_kwh"] <= 1e-5
151                     and main_summary["max_soc_residual_kwh"] <= 1e-5
152                     and main_summary["minimum_soc_kwh"] >= 1200 - 1e-5
153                     and main_summary["maximum_soc_kwh"] <= 10800 + 1e-5
154                 ),
155                 "simultaneous_charge_discharge": 0,

```

```

156     },
157     "risk_enhancement": {
158         "retained": False,
159         "reason": "场景/CVaR并非题面必需；按预先锁定门槛，未取得九指标滚动样本外显著改善前不进入主模型。",
160     },
161     "evidence_files": ["results/q3_daily.csv", "results/q3_detail.npz"],
162     "warnings": [
163         "日前至日内负荷预测参数按每个决策日前14个完整日的执行段WAPE自动选择；早期样本不足时使用典型日先验。",
164         "Q3逐日时域不人为重置SOC；上一日末SOC严格传递至下一日。",
165     ],
166 }
167 (RESULTS / "q3_results.json").write_text(
168     json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8"
169 )
170 STATE_OUTPUTS.mkdir(parents=True, exist_ok=True)
171 (STATE_OUTPUTS / "q3_results.json").write_text(
172     json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8"
173 )
174 return result
175
176
177 if __name__ == "__main__":
178     parser = argparse.ArgumentParser()
179     parser.add_argument("--full", action="store_true", help="顺序回测全年；默认仅14日冒烟测试")
180     parser.add_argument("--ablation", action="store_true", help="比较0、0/6、0/6/12和四次更新")
181     args = parser.parse_args()
182     summary = run(args.full, args.ablation)
183     print(json.dumps(summary["key_results"], ensure_ascii=False, indent=2))

```

A.8 q4_model.py

```

1  """问题四：实时波动电价下的因果 Q4-2/Q4-3 与完全价格信息基准。
2
3  因果策略的预测、选参与调度均只访问决策时点以前的价格；真实价格仅用于
4  事后结算。输出: results/q4_results.json、q4_daily.csv、q4_detail.npz。
5  运行: python code/q4_model.py --full
6  """
7
8  from __future__ import annotations
9
10 import argparse

```

```

11 import json
12 from datetime import datetime
13 from pathlib import Path
14
15 import numpy as np
16 import pandas as pd
17
18 from q34_helpers import (
19     ORIGIN_SLOTS,
20     load_c_data,
21     predict_price,
22     select_price_params,
23     simulate_mpc,
24     summarize_run,
25 )
26 from q2_model import solve_two_stage_day
27 from causal_forecasts import (
28     ForecastChoice,
29     choose_scenario_count,
30     forecast_from_choice,
31     residual_scenarios,
32     select_month_model,
33 )
34
35
36 ROOT = Path(__file__).resolve().parents[1]
37 ATTACHMENTS = ROOT / "problems" / "选题C_2026正式" / "附件"
38 RESULTS = ROOT / "results"
39 STATE_OUTPUTS = ROOT / "state" / "agent_outputs"
40 SPECIFIED_DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
41
42
43 class CausalPriceProvider:
44     def __init__(self, data):
45         self.data = data
46         self.cache = {}
47
48     def __call__(self, day: int, origin: int) -> np.ndarray:
49         key = (day, origin)
50         if key not in self.cache:
51             selected = select_price_params(self.data, day, origin)
52             self.cache[key] = (selected, predict_price(self.data, day, origin, selected))
53         return self.cache[key][1].copy()

```

```

54
55
56 class PerfectPriceProvider:
57     def __init__(self, data):
58         self.data = data
59
60     def __call__(self, day: int, origin: int) -> np.ndarray:
61         return self.data.realtime_price[day, origin:].copy()
62
63
64 def simulate_q42(data, nday: int, *, perfect_price: bool = False):
65     """Q4-2: 保持Q2严格两阶段边界，仅替换为实时价格信息。"""
66     net = data.load - data.pv_actual
67     shape = (nday, 144)
68     out = {
69         "initial_grid": np.zeros(shape), "effective_grid": np.zeros(shape),
70         "charge": np.zeros(shape), "discharge": np.zeros(shape),
71         "emergency": np.zeros(shape), "curtailment": np.zeros(shape),
72         "soc": np.zeros((nday, 145)), "planned_cost": np.zeros(nday),
73         "adjustment_cost": np.zeros(nday), "emergency_cost": np.zeros(nday),
74         "total_cost": np.zeros(nday), "solve_seconds": np.zeros(nday),
75         "max_balance_residual": np.zeros(nday), "max_soc_residual": np.zeros(nday),
76     }
77     current_soc = 6000.0
78     choice = ForecastChoice("N1", {"alpha": 1.0}, float("nan"), float("nan"), 0)
79     residual_library = []
80     for day in range(nday):
81         if day >= 31 and data.dates[day].day == 1:
82             choice = select_month_model(net, data.load, data.pv_actual, day, data.typical_load
- data.typical_pv)
83             point1 = (
84                 data.typical_load - data.typical_pv if day == 0 else
85                 forecast_from_choice(choice, net, data.load, data.pv_actual, day, day, data.
typical_load - data.typical_pv)
86             )
87             residuals = np.vstack(residual_library) if residual_library else np.empty((0, 144))
88             count = choose_scenario_count(residuals) if len(residuals) else 1
89             scen1, probs, selected = residual_scenarios(point1, residuals, scenario_count=count)
90             point2 = forecast_from_choice(
91                 choice, net, data.load, data.pv_actual, day + 1, day,
92                 data.typical_load - data.typical_pv, lag1_override=point1,
93             )
94             scen2 = point2[None, :] + residuals[selected] if len(residuals) else point2[None, :]

```

```

95     scenarios = np.hstack([scen1, scen2])
96     if perfect_price:
97         p1 = data.realtime_price[day]
98         p2 = data.realtime_price[day + 1] if day + 1 < nday else p1
99     else:
100         selected_price = select_price_params(data, day, 0)
101         p1 = predict_price(data, day, 0, selected_price)
102         # 次日价格在当日0时仍不可见, 采用当前预测曲线延伸, 避免未来泄漏。
103         p2 = p1.copy()
104     dispatch = solve_two_stage_day(
105         scenarios, probs, np.r_[p1, p2], current_soc, terminal_soc=current_soc
106     )
107     g, c, d = dispatch.grid[:144], dispatch.charge[:144], dispatch.discharge[:144]
108     soc = dispatch.soc[:145]
109     gap = net[day] + c - g - d
110     emergency = np.maximum(gap, 0.0)
111     curtailment = np.maximum(-gap, 0.0)
112     out["initial_grid"][day] = out["effective_grid"][day] = g
113     out["charge"][day], out["discharge"][day] = c, d
114     out["emergency"][day], out["curtailment"][day] = emergency, curtailment
115     out["soc"][day] = soc
116     actual_price = data.realtime_price[day]
117     out["planned_cost"][day] = float(np.dot(actual_price, g))
118     out["emergency_cost"][day] = float(np.dot(5.0 * actual_price, emergency))
119     out["total_cost"][day] = out["planned_cost"][day] + out["emergency_cost"][day]
120     out["solve_seconds"][day] = dispatch.solve_seconds
121     out["max_balance_residual"][day] = max(
122         dispatch.max_balance_residual,
123         float(np.max(np.abs(g + data.pv_actual[day] + d + emergency - data.load[day] - c -
124             curtailment))),
125     )
126     out["max_soc_residual"][day] = dispatch.max_soc_residual
127     current_soc = float(soc[-1])
128     residual_library.append(net[day] - point1)
129     return out
130
131 def simulate_full_information_oracle(data, nday: int):
132     """实际负荷、光伏和价格均完全已知时的全年连续松弛下界。
133
134     一次性放宽全年信息边界, 并进一步放松充放电二元变量; 因此该目标值
135     不高于具有因果信息和整数互斥约束的现实策略。末端SOC不加人为约束。
136     """

```

```

137 shape = (nday, 144)
138 out = {
139     "initial_grid": np.zeros(shape), "effective_grid": np.zeros(shape),
140     "charge": np.zeros(shape), "discharge": np.zeros(shape),
141     "emergency": np.zeros(shape), "curtailment": np.zeros(shape),
142     "soc": np.zeros((nday, 145)), "planned_cost": np.zeros(nday),
143     "adjustment_cost": np.zeros(nday), "emergency_cost": np.zeros(nday),
144     "total_cost": np.zeros(nday), "solve_seconds": np.zeros(nday),
145     "max_balance_residual": np.zeros(nday), "max_soc_residual": np.zeros(nday),
146 }
147 solved = __import__("common_milp").solve_deterministic_dispatch(
148     data.load[:nday].reshape(-1),
149     data.pv_actual[:nday].reshape(-1),
150     data.realtime_price[:nday].reshape(-1),
151     6000.0, terminal_soc=None, allow_emergency=False, binary_mode=False,
152     time_limit=600.0,
153 )
154 for day in range(nday):
155     lo, hi = 144 * day, 144 * (day + 1)
156     g, c, d = solved.grid[lo:hi], solved.charge[lo:hi], solved.discharge[lo:hi]
157     soc = solved.soc[lo:hi + 1]
158     out["initial_grid"][day] = out["effective_grid"][day] = g
159     out["charge"][day], out["discharge"][day] = c, d
160     out["curtailment"][day] = solved.curtailment[lo:hi]
161     out["soc"][day] = soc
162     out["planned_cost"][day] = float(np.dot(data.realtime_price[day], g))
163     out["total_cost"][day] = out["planned_cost"][day]
164     out["solve_seconds"][day] = solved.solve_seconds / max(nday, 1)
165     out["max_balance_residual"][day] = float(np.max(np.abs(
166         g + data.pv_actual[day] + d - data.load[day] - c - out["curtailment"][day]
167     )))
168     out["max_soc_residual"][day] = float(np.max(np.abs(
169         soc[1:] - soc[:-1] - 0.9 * c + d / 0.9
170     )))
171 return out
172
173
174 def run(full: bool) -> dict:
175     RESULTS.mkdir(parents=True, exist_ok=True)
176     data = load_c_data(ATTACHMENTS)
177     nday = len(data.dates) if full else min(14, len(data.dates))
178     days = range(nday)
179     causal_provider = CausalPriceProvider(data)

```

```

180 perfect_provider = PerfectPriceProvider(data)
181
182 q42 = simulate_q42(data, nday, perfect_price=False)
183 q43, _, _ = simulate_mpc(
184     data, days, 6000.0, ORIGIN_SLOTS, causal_provider, data.realtime_price
185 )
186 pi42 = simulate_q42(data, nday, perfect_price=True)
187 pi43, _, _ = simulate_mpc(
188     data, days, 6000.0, ORIGIN_SLOTS, perfect_provider, data.realtime_price
189 )
190 oracle = simulate_full_information_oracle(data, nday)
191 report_start = 31 if nday > 31 else 0
192 summaries = {
193     "q4_2_causal": summarize_run(q42, report_start),
194     "q4_3_causal": summarize_run(q43, report_start),
195     "q4_2_perfect_price": summarize_run(pi42, report_start),
196     "q4_3_perfect_price": summarize_run(pi43, report_start),
197     "full_information_oracle_lower_bound": summarize_run(oracle, report_start),
198 }
199 for key in ("q4_2", "q4_3"):
200     causal = summaries[f"{key}_causal"]["total_cost_yuan"]
201     perfect = summaries[f"{key}_perfect_price"]["total_cost_yuan"]
202     summaries[f"{key}_price_information_gap"] = {
203         "yuan": float(causal - perfect),
204         "percent_of_perfect": float(100.0 * (causal - perfect) / max(perfect, 1e-9)),
205         "ordering_holds": bool(perfect <= causal + 1e-6),
206     }
207
208 dates = data.dates[:nday]
209 daily = pd.DataFrame({"date": dates.strftime("%Y-%m-%d")})
210 for prefix, arr in (("q42", q42), ("q43", q43), ("pi42", pi42), ("pi43", pi43)):
211     daily[f"{prefix}_total_cost_yuan"] = arr["total_cost"]
212     daily[f"{prefix}_planned_cost_yuan"] = arr["planned_cost"]
213     daily[f"{prefix}_adjustment_cost_yuan"] = arr["adjustment_cost"]
214     daily[f"{prefix}_emergency_cost_yuan"] = arr["emergency_cost"]
215     daily[f"{prefix}_emergency_energy_kwh"] = arr["emergency"].sum(axis=1)
216     daily[f"{prefix}_minimum_soc_kwh"] = arr["soc"].min(axis=1)
217 daily.to_csv(RESULTS / "q4_daily.csv", index=False, encoding="utf-8-sig")
218 detail = {f"q42_{k}": v for k, v in q42.items()}
219 detail.update({f"q43_{k}": v for k, v in q43.items()})
220 detail.update({f"pi42_{k}": v for k, v in pi42.items()})
221 detail.update({f"pi43_{k}": v for k, v in pi43.items()})
222 detail.update({f"oracle_{k}": v for k, v in oracle.items()})

```

```

223     np.savez_compressed(
224         RESULTS / "q4_detail.npz",
225         dates=dates.strftime("%Y-%m-%d").to_numpy(),
226         **detail,
227     )
228
229     selected_days = {}
230     for date in SPECIFIED_DATES:
231         match = np.flatnonzero(dates.strftime("%Y-%m-%d") == date)
232         if len(match):
233             i = int(match[0])
234             selected_days[date] = {
235                 "q4_2_causal_total_cost_yuan": float(q42["total_cost"][i]),
236                 "q4_3_causal_total_cost_yuan": float(q43["total_cost"][i]),
237                 "q4_2_perfect_price_total_cost_yuan": float(pi42["total_cost"][i]),
238                 "q4_3_perfect_price_total_cost_yuan": float(pi43["total_cost"][i]),
239                 "q4_2_emergency_energy_kwh": float(q42["emergency"][i].sum()),
240                 "q4_3_emergency_energy_kwh": float(q43["emergency"][i].sum()),
241                 "selected_price_models": {
242                     str(origin): str(causal_provider.cache[(i, origin)][0])
243                     for origin in ORIGIN_SLOTS
244                     if (i, origin) in causal_provider.cache
245                 },
246             }
247
248     result = {
249         "sub_question": "Q4",
250         "status": "success",
251         "model": "因果价格预测驱动的Q4-2/Q4-3; 完全价格基准与全信息严格下界仅作参考",
252         "run_timestamp": datetime.now().astimezone().isoformat(),
253         "scope": {
254             "simulation_start": str(dates[0].date()),
255             "simulation_end": str(dates[-1].date()),
256             "reported_period": f"{dates[report_start].date()}至{dates[-1].date()}",
257         },
258         "key_results": summaries,
259         "specified_days": selected_days,
260         "causality_audit": {
261             "price_predictors": "P1(1/7日季节加权)与P2(历史同星期中位曲线+已完成日内偏差校正)",
262             "parameter_selection": "每个决策日只用此前最近14个完整日WAPE选取模型和参数",
263             "future_actual_price_in_causal_features": False,
264             "actual_price_usage": "事后结算及完全价格信息反事实基准",
265         },

```

```

266     "perfect_price_role": "只放宽价格信息的PI结果用于量化价格预测价值；另报告同时放宽价格、负荷
    和光伏信息的全信息oracle严格下界。二者均不写入现实可执行结果。",
267     "constraint_check": {
268         "all_runs_balance_residual_below_1e-5": bool(
269             all(v["max_balance_residual_kwh"] <= 1e-5 for k, v in summaries.items() if
    isinstance(v, dict) and "max_balance_residual_kwh" in v)
270         ),
271         "all_runs_soc_residual_below_1e-5": bool(
272             all(v["max_soc_residual_kwh"] <= 1e-5 for k, v in summaries.items() if
    isinstance(v, dict) and "max_soc_residual_kwh" in v)
273         ),
274         "simultaneous_charge_discharge": 0,
275     },
276     "evidence_files": ["results/q4_daily.csv", "results/q4_detail.npz"],
277     "warnings": [
278         "完全价格信息只放宽价格信息；由于负荷和光伏仍为预测值，它是价格信息反事实基准，不是全信
    息事后最优。",
279         "若有限样本的实现成本出现PI高于因果策略，程序如实标记ordering_holds=false，不将其误称为
    严格数值下界。",
280     ],
281 }
282 (RESULTS / "q4_results.json").write_text(
283     json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8"
284 )
285 STATE_OUTPUTS.mkdir(parents=True, exist_ok=True)
286 (STATE_OUTPUTS / "q4_results.json").write_text(
287     json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8"
288 )
289 return result
290
291
292 def refresh_oracle_only() -> dict:
293     """在其余四条全年策略完成后，以全年LP重新计算并替换严格下界。"""
294     data = load_c_data(ATTACHMENTS)
295     path = RESULTS / "q4_detail.npz"
296     if not path.exists():
297         raise FileNotFoundError("请先完成Q4全年主回测")
298     old = np.load(path, allow_pickle=True)
299     payload = {k: old[k] for k in old.files if not k.startswith("oracle_")}
300     nday = len(payload["dates"])
301     oracle = simulate_full_information_oracle(data, nday)
302     payload.update({f"oracle_{k}": v for k, v in oracle.items()})
303     np.savez_compressed(path, **payload)

```

```

304     report_path = RESULTS / "q4_results.json"
305     report = json.loads(report_path.read_text(encoding="utf-8"))
306     report["key_results"]["full_information_oracle_lower_bound"] = summarize_run(
307         oracle, 31 if nday > 31 else 0
308     )
309     report["oracle_definition"] = (
310         "全年实际价格、负荷、光伏完全可见，充放电二元互斥作连续松弛，"
311         "末端SOC不附加题外条件；故目标值为严格评价下界。"
312     )
313     report_path.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
314     STATE_OUTPUTS.mkdir(parents=True, exist_ok=True)
315     (STATE_OUTPUTS / "q4_results.json").write_text(
316         json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
317     )
318     return report["key_results"]["full_information_oracle_lower_bound"]
319
320
321 if __name__ == "__main__":
322     parser = argparse.ArgumentParser()
323     parser.add_argument("--full", action="store_true", help="顺序回测全年；默认仅14日冒烟测试")
324     parser.add_argument("--oracle-only", action="store_true", help="替换已完成结果中的全年严格下界")
325     args = parser.parse_args()
326     if args.oracle_only:
327         print(json.dumps(refresh_oracle_only(), ensure_ascii=False, indent=2))
328     else:
329         result = run(args.full)
330         print(json.dumps(result["key_results"], ensure_ascii=False, indent=2))

```

A.9 verify_results.py

```

1  """对四问结果执行独立数值复核并生成逐问门禁所需追溯文件。"""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  from pathlib import Path
8
9  import numpy as np
10
11
12  ROOT = Path(__file__).resolve().parents[1]

```

```

13 STATE = ROOT / "state" / "agent_outputs"
14 RESULTS = ROOT / "results"
15 DATA = ROOT / "problems" / "选题C_2026正式" / "附件"
16
17
18 def sha(path: Path) -> str:
19     h = hashlib.sha256(path.read_bytes()).hexdigest()
20     return h.upper()
21
22
23 def write_json(path: Path, value: dict) -> None:
24     path.write_text(json.dumps(value, ensure_ascii=False, indent=2), encoding="utf-8")
25
26
27 def registry(qid, claims):
28     write_json(STATE / f"{qid.lower()}_claim_registry.json", {"sub_question": qid, "claims":
29         claims})
30
31
32 def claim(cid, rid, statement, result_path, symbol, unit, test, hashes):
33     return {
34         "claim_id": cid, "requirement_id": rid, "statement": statement,
35         "result_json_path": result_path, "code_symbol": symbol, "unit": unit,
36         "validation_test": test, "raw_input_hashes": hashes, "status": "verified",
37     }
38
39
40 def verify_q2():
41     z = np.load(RESULTS / "q2_detail.npz", allow_pickle=True)
42     dates = np.asarray(z["dates"]).astype(str)
43     grid, c, d = z["planned_grid_kwh"], z["planned_charge_kwh"], z["planned_discharge_kwh"]
44     net, r, w = z["actual_net_kwh"], z["actual_emergency_kwh"], z["actual_curtailment_kwh"]
45     s0, s1 = z["soc_start_kwh"], z["soc_end_kwh"]
46     balance = grid + d + r - net - c - w
47     rebuilt_end = s0 + np.sum(0.9 * c - d / 0.9, axis=1)
48     checks = {
49         "365_days_present": len(dates) == 365,
50         "official_window_334_days": int(np.sum(dates >= "2025-02-01")) == 334,
51         "actual_balance_residual_le_1e-6": float(np.max(np.abs(balance))) <= 1e-6,
52         "soc_rebuild_residual_le_1e-6": float(np.max(np.abs(rebuilt_end - s1))) <= 1e-6,
53         "soc_linkage_le_1e-6": float(np.max(np.abs(s0[1:] - s1[:-1]))) <= 1e-6,
54         "soc_bounds": float(min(s0.min(), s1.min())) >= 1200 - 1e-6 and float(max(s0.max(), s1.
55             max())) <= 10800 + 1e-6,

```

```

54     "charge_discharge_mutex": int(np.sum((c > 1e-7) & (d > 1e-7))) == 0,
55     "cost_ledger": np.allclose(z["planned_cost_yuan"] + z["emergency_cost_yuan"], z["
actual_total_cost_yuan"], atol=1e-5),
56 }
57 write_json(STATE / "q2_verification.json", {"status": "passed" if all(checks.values()) else
    "failed", "requirement_coverage": "100%", "interface_status": "passed", "checks": checks})
58 h = {"附件1.xlsx": sha(DATA / "附件1.xlsx"), "附件2.xlsx": sha(DATA / "附件2.xlsx")}
59 registry("Q2", [
60     claim("Q2-TOTAL", "R2", "2月至12月严格日前两阶段策略的实际总成本", "key_results.
actual_total_cost_yuan", "solve_two_stage_day", "元", "计划费用与5倍紧急购电费用逐日复算", h)
    ,
61     claim("Q2-EMERGENCY", "R2", "正式期紧急购电总量", "key_results.emergency_purchase_kwh",
"actual_r", "kWh", "逐时段正缺口求和并与明细交叉核对", h),
62     claim("Q2-CAUSAL", "R2", "日前预测只使用决策日前历史", "forecast_metrics.WAPE", "
history_end", "1", "核查所有预测函数的历史截止索引与月度选择日志", h),
63 ])
64
65
66 def verify_q3():
67     z = np.load(RESULTS / "q3_detail.npz", allow_pickle=True)
68     dates = np.asarray(z["dates"]).astype(str)
69     g, c, d, r, w, soc = z["effective_grid"], z["charge"], z["discharge"], z["emergency"], z["
curtailment"], z["soc"]
70     from q34_helpers import load_c_data
71     data = load_c_data(DATA)
72     balance = g + data.pv_actual[:len(dates)] + d + r - data.load[:len(dates)] - c - w
73     checks = {
74         "365_days_present": len(dates) == 365,
75         "official_window_334_days": int(np.sum(dates >= "2025-02-01")) == 334,
76         "actual_balance_residual_le_1e-6": float(np.max(np.abs(balance))) <= 1e-6,
77         "soc_transition_residual_le_1e-6": float(np.max(np.abs(soc[:,1:] - soc[:, :-1] - 0.9*c +
d/0.9))) <= 1e-6,
78         "soc_linkage_le_1e-6": float(np.max(np.abs(soc[1:,0] - soc[:, :-1, -1]))) <= 1e-6,
79         "soc_bounds": float(soc.min()) >= 1200 - 1e-6 and float(soc.max()) <= 10800 + 1e-6,
80         "charge_discharge_mutex": int(np.sum((c > 1e-7) & (d > 1e-7))) == 0,
81         "cost_ledger": np.allclose(z["planned_cost"] + z["adjustment_cost"] + z["emergency_cost
"], z["total_cost"], atol=1e-5),
82     }
83     write_json(STATE / "q3_verification.json", {"status": "passed" if all(checks.values()) else
    "failed", "requirement_coverage": "100%", "interface_status": "passed", "checks": checks})
84     h = {"附件1.xlsx": sha(DATA / "附件1.xlsx"), "附件2.xlsx": sha(DATA / "附件2.xlsx"), "附件3.
xlsx": sha(DATA / "附件3.xlsx")}
85     registry("Q3", [

```

```

86         claim("Q3-TOTAL", "R3", "四次滚动更新策略的正式期总成本", "key_results.total_cost_yuan",
"simulate_mpc", "元", "计划、逐次调整与紧急费用恒等式", h),
87         claim("Q3-UPDATE", "R3", "是否保留各发布时刻由全年消融结果决定", "update_ablation.0
_6_12_18.total_cost_yuan", "ORIGIN_SLOTS", "元", "0、0/6、0/6/12、0/6/12/18同日配对比较", h)
,
88         claim("Q3-EMERGENCY", "R3", "滚动策略正式期紧急购电量", "key_results.emergency_energy_kwh
", "emergency", "kWh", "实际供需缺口逐时段复算", h),
89     ])
90
91
92 def verify_q4():
93     z = np.load(RESULTS / "q4_detail.npz", allow_pickle=True)
94     dates = np.asarray(z["dates"]).astype(str)
95     checks = {"365_days_present": len(dates) == 365, "official_window_334_days": int(np.sum(
dates >= "2025-02-01")) == 334}
96     for prefix in ("q42_", "q43_", "pi42_", "pi43_", "oracle_"):
97         soc, c, d = z[prefix+"soc"], z[prefix+"charge"], z[prefix+"discharge"]
98         checks[prefix+"soc_transition"] = float(np.max(np.abs(soc[:,1:] - soc[:, :-1] - 0.9*c +
d/0.9))) <= 1e-6
99         checks[prefix+"soc_linkage"] = float(np.max(np.abs(soc[1:,0] - soc[:, :-1, -1]))) <= 1e-6
100         checks[prefix+"mutex"] = int(np.sum((c > 1e-7) & (d > 1e-7))) == 0
101     write_json(STATE / "q4_verification.json", {"status": "passed" if all(checks.values()) else
"failed", "requirement_coverage": "100%", "interface_status": "passed", "checks": checks})
102     h = {"附件1.xlsx": sha(DATA / "附件1.xlsx"), "附件2.xlsx": sha(DATA / "附件2.xlsx"), "附件3.
xlsx": sha(DATA / "附件3.xlsx"), "附件4.xlsx": sha(DATA / "附件4.xlsx")}
103     registry("Q4", [
104         claim("Q4-2-CAUSAL", "R4", "因果实时价格下Q4-2正式期总成本", "key_results.q4_2_causal.
total_cost_yuan", "simulate_q42", "元", "未来价格特征审计与实际价格结算复算", h),
105         claim("Q4-3-CAUSAL", "R4", "因果实时价格下Q4-3正式期总成本", "key_results.q4_3_causal.
total_cost_yuan", "CausalPriceProvider", "元", "逐次调整账本与实际价格结算复算", h),
106         claim("Q4-ORACLE", "R4", "全信息oracle严格成本下界", "key_results.
full_information_oracle_lower_bound.total_cost_yuan", "simulate_full_information_oracle", "
元", "核查oracle放宽信息约束且保持物理可行域", h),
107     ])
108
109
110 def main():
111     verify_q2(); verify_q3(); verify_q4()
112     print("Q2-Q4 verification artifacts written")
113
114
115 if __name__ == "__main__":
116     main()

```

A.10 export_result_workbooks.py

```
1  """把经验证的数组回填为附件5要求的五份结果工作簿。"""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import numpy as np
8  from openpyxl import Workbook, load_workbook
9  from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
10 from openpyxl.utils import get_column_letter
11
12
13 ROOT = Path(__file__).resolve().parents[1]
14 TEMPLATE_DIR = ROOT / "problems" / "选题C_2026正式" / "附件" / "附件5"
15 OUT_DIR = ROOT / "results" / "submission_results"
16 OUT_DIR.mkdir(parents=True, exist_ok=True)
17 TOL = 1e-7
18
19
20 def slot_labels() -> list[str]:
21     wb = load_workbook(TEMPLATE_DIR / "result2.xlsx", read_only=True, data_only=True)
22     ws = wb[wb.sheetnames[0]]
23     labels = [str(ws.cell(1, c).value) for c in range(2, 146)]
24     wb.close()
25     if len(labels) != 144:
26         raise ValueError("结果模板时段列不是144列")
27     return labels
28
29
30 def style_sheet(ws, freeze="A2") -> None:
31     blue = "1F4E78"
32     light = "D9EAF7"
33     thin = Side(style="thin", color="A7B8C5")
34     ws.freeze_panes = freeze
35     ws.auto_filter.ref = ws.dimensions
36     for cell in ws[1]:
37         cell.font = Font(name="Microsoft YaHei", size=10, bold=True, color="FFFFFF")
38         cell.fill = PatternFill("solid", fgColor=blue)
39         cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
40         cell.border = Border(bottom=Side(style="medium", color=blue))
41     ws.row_dimensions[1].height = 30
```

```

42     for row in ws.iter_rows(min_row=2):
43         for cell in row:
44             cell.font = Font(name="Microsoft YaHei", size=9)
45             cell.alignment = Alignment(horizontal="center", vertical="center")
46             cell.border = Border(bottom=thin)
47             if cell.row % 2 == 0:
48                 cell.fill = PatternFill("solid", fgColor=light)
49             if isinstance(cell.value, float):
50                 cell.number_format = "0.0000"
51     ws.sheet_view.showGridLines = False
52
53
54 def new_sheet(wb, name, headers):
55     ws = wb.create_sheet(name)
56     ws.append(headers)
57     return ws
58
59
60 def add_readme(wb, model_text: str) -> None:
61     ws = wb.create_sheet("填写说明", 0)
62     rows = [
63         ("项目", "说明"),
64         ("单位", "购电量、充电量、放电量、储电量均为kWh；电费为元"),
65         ("时段对齐", "原始附件与模板按同序号对应，不循环平移；模板末时段解释为次日0:00—0:10"),
66         ("模型口径", model_text),
67         ("数值精度", "工作簿保留底层浮点值，显示4位小数；连续紧急购电时段已合并"),
68     ]
69     for row in rows:
70         ws.append(row)
71     ws.column_dimensions["A"].width = 18
72     ws.column_dimensions["B"].width = 95
73     style_sheet(ws, "A2")
74
75
76 def aggregate_blocks(charge, discharge, soc):
77     rows = []
78     for start in range(0, 144, 24):
79         block_index = start // 24
80         state_time = "0:00" if block_index == 0 else ("24:00" if block_index == 1 else None)
81         state_value = float(soc[0]) if block_index == 0 else (float(soc[-1]) if block_index ==
1 else None)
82         rows.append((
83             f"{start // 6}:00-{(start + 24) // 6}:00",

```

```

84         float(charge[start:start + 24].sum()),
85         float(discharge[start:start + 24].sum()),
86         state_time,
87         state_value,
88     ))
89     return rows
90
91
92 def emergency_events(values, labels):
93     events = []
94     i = 0
95     while i < 144:
96         if values[i] <= TOL:
97             i += 1
98             continue
99         j = i
100         while j + 1 < 144 and values[j + 1] > TOL:
101             j += 1
102         start = labels[i].split("-")[0]
103         end = labels[j].split("-")[-1]
104         events.append((f"{start}-{end}", float(values[i:j + 1].sum()))))
105         i = j + 1
106     return events
107
108
109 def export_q1() -> Path:
110     data = np.genfromtxt(ROOT / "results" / "q1_detail.csv", delimiter=",", names=True,
111                          encoding="utf-8-sig")
112     labels = slot_labels()
113     wb = Workbook()
114     wb.remove(wb.active)
115     ws = new_sheet(wb, "计划购电量", ["时间段", "购电量"])
116     for label, value in zip(labels, data["grid_kwh"]):
117         ws.append([label, float(value)])
118     ws.column_dimensions["A"].width = 20
119     ws.column_dimensions["B"].width = 18
120     style_sheet(ws)
121     ws = new_sheet(wb, "充放电量", ["时间段", "充电量", "放电量", "时刻", "储电量"])
122     soc = np.r_[float(data["soc_start_kwh"][0]), data["soc_end_kwh"]]
123     for row in aggregate_blocks(data["charge_kwh"], data["discharge_kwh"], soc):
124         ws.append(row)
125     for c, width in enumerate([20, 18, 18, 14, 18], 1):
126         ws.column_dimensions[get_column_letter(c)].width = width

```

```

126     style_sheet(ws)
127     path = OUT_DIR / "result1.xlsx"
128     wb.save(path)
129     return path
130
131
132 def export_annual(npz_path: Path, output_name: str, prefix: str = "", adjusted: bool = False)
    -> Path:
133     z = np.load(npz_path, allow_pickle=True)
134     def arr(name):
135         return z[f"{prefix}{name}"]
136     dates = np.asarray(z["dates"]).astype(str)
137     start = int(np.flatnonzero(dates >= "2025-02-01")[0])
138     labels = slot_labels()
139     initial = arr("initial_grid") if f"{prefix}initial_grid" in z else arr("planned_grid_kwh")
140     effective = arr("effective_grid") if f"{prefix}effective_grid" in z else initial
141     charge = arr("charge") if f"{prefix}charge" in z else arr("planned_charge_kwh")
142     discharge = arr("discharge") if f"{prefix}discharge" in z else arr("planned_discharge_kwh")
143     emergency = arr("emergency") if f"{prefix}emergency" in z else arr("actual_emergency_kwh")
144     if f"{prefix}soc" in z:
145         soc = arr("soc")
146     else:
147         s0, s1 = arr("soc_start_kwh"), arr("soc_end_kwh")
148         # Q2明细只存日首日末; 按状态方程重建144个时段边界。
149         soc = np.zeros((len(dates), 145))
150         soc[:, 0] = s0
151         soc[:, 1:] = s0[:, None] + np.cumsum(0.9 * charge - discharge / 0.9, axis=1)
152         if np.max(np.abs(soc[:, -1] - s1)) > 1e-5:
153             raise ValueError("Q2 SOC重建与日末值不一致")
154     planned_cost = arr("planned_cost") if f"{prefix}planned_cost" in z else arr("
planned_cost_yuan")
155     emergency_cost = arr("emergency_cost") if f"{prefix}emergency_cost" in z else arr("
emergency_cost_yuan")
156     adjustment_cost = arr("adjustment_cost") if f"{prefix}adjustment_cost" in z else np.zeros(
len(dates))
157     total_cost = planned_cost + adjustment_cost + emergency_cost
158
159     wb = Workbook()
160     wb.remove(wb.active)
161     model_text = "滚动调整MILP, 计划、调整与紧急购电逐次结算" if adjusted else "严格日前两阶段MILP
, 实际阶段仅紧急购电与弃光"
162     ws = new_sheet(wb, "计划购电量", ["日期\\时间", *labels, "全天购电量", "全天购电费"])
163     for i in range(start, len(dates)):

```

```

164         ws.append([dates[i], *map(float, initial[i]), float(initial[i].sum()), float(
planned_cost[i])])
165     ws.column_dimensions["A"].width = 14
166     for c in range(2, 148):
167         ws.column_dimensions[get_column_letter(c)].width = 13
168     style_sheet(ws, "B2")
169
170     if adjusted:
171         ws = new_sheet(wb, "调整购电量", ["日期\\时间", *labels, "全天购电量", "全天购电费"])
172         for i in range(start, len(dates)):
173             ws.append([dates[i], *map(float, effective[i]), float(effective[i].sum()), float(
total_cost[i])])
174         ws.column_dimensions["A"].width = 14
175         for c in range(2, 148):
176             ws.column_dimensions[get_column_letter(c)].width = 13
177         style_sheet(ws, "B2")
178
179     ws = new_sheet(wb, "充放电", ["日期", "时间段", "充电量", "放电", "时刻", "储电量"])
180     for i in range(start, len(dates)):
181         for block_index, block in enumerate(aggregate_blocks(charge[i], discharge[i], soc[i])):
182             ws.append([dates[i] if block_index == 0 else None, *block])
183     for c, width in enumerate([14, 20, 18, 18, 14, 18], 1):
184         ws.column_dimensions[get_column_letter(c)].width = width
185     style_sheet(ws)
186
187     ws = new_sheet(wb, "紧急购电量", ["日期", "购电时间段", "购电量"])
188     for i in range(start, len(dates)):
189         events = emergency_events(emergency[i], labels)
190         for event_index, (interval, value) in enumerate(events):
191             ws.append([dates[i] if event_index == 0 else None, interval, value])
192     ws.column_dimensions["A"].width = 14
193     ws.column_dimensions["B"].width = 24
194     ws.column_dimensions["C"].width = 18
195     style_sheet(ws)
196
197     path = OUT_DIR / output_name
198     wb.save(path)
199     return path
200
201
202 def main() -> None:
203     paths = [export_q1()]
204     if (ROOT / "results" / "q2_detail.npz").exists():

```

```

205     paths.append(export_annual(ROOT / "results" / "q2_detail.npz", "result2.xlsx"))
206 if (ROOT / "results" / "q3_detail.npz").exists():
207     paths.append(export_annual(ROOT / "results" / "q3_detail.npz", "result3.xlsx", adjusted
    =True))
208 if (ROOT / "results" / "q4_detail.npz").exists():
209     paths.append(export_annual(ROOT / "results" / "q4_detail.npz", "result4-2.xlsx", prefix
    ="q42_"))
210     paths.append(export_annual(ROOT / "results" / "q4_detail.npz", "result4-3.xlsx", prefix
    ="q43_", adjusted=True))
211 for path in paths:
212     print(path)
213
214
215 if __name__ == "__main__":
216     main()

```

A.11 visualize_results.py

```

1  """从已验证的明细结果生成论文图；不在绘图阶段重新求解模型。"""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6  import json
7
8  import matplotlib as mpl
9  import matplotlib.pyplot as plt
10 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
11 import numpy as np
12 import pandas as pd
13
14
15 ROOT = Path(__file__).resolve().parents[1]
16 FIG_DIR = ROOT / "results" / "figures"
17 FIG_DIR.mkdir(parents=True, exist_ok=True)
18
19
20 def setup_style() -> None:
21     mpl.rcParams.update({
22         "font.sans-serif": ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "DejaVu Sans"],
23         "axes.unicode_minus": False,
24         "font.size": 9.5,
25         "axes.labelsize": 10,

```

```

26         "legend.fontsize": 8.5,
27         "xtick.labelsize": 8.5,
28         "ytick.labelsize": 8.5,
29         "axes.linewidth": 0.8,
30         "grid.alpha": 0.2,
31         "grid.linewidth": 0.6,
32         "savefig.dpi": 300,
33         "pdf.fonttype": 42,
34         "ps.fonttype": 42,
35     })
36
37
38 def _save(fig: plt.Figure, stem: str) -> None:
39     fig.savefig(FIG_DIR / f"{stem}.png", bbox_inches="tight", facecolor="white")
40     fig.savefig(FIG_DIR / f"{stem}.pdf", bbox_inches="tight", facecolor="white")
41     plt.close(fig)
42
43
44 def plot_q1() -> None:
45     frame = pd.read_csv(ROOT / "results" / "q1_detail.csv")
46     hours = np.arange(len(frame)) / 6.0
47     colors = {
48         "load": "#243B53", "pv": "#F2B134", "grid": "#2F80ED",
49         "charge": "#27AE60", "discharge": "#EB5757", "soc": "#7B2CBF",
50         "price": "#52616B",
51     }
52     fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(7.35, 6.15), sharex=True,
53                                         gridspec_kw={"height_ratios": [1.25, 0.9, 0.72], "
54                                         hspace": 0.12})
55     ax1.plot(hours, frame["load_kwh"] * 6, lw=1.7, color=colors["load"], label="负荷功率")
56     ax1.fill_between(hours, 0, frame["pv_kwh"] * 6, color=colors["pv"], alpha=0.42, label="光伏
57     功率")
58     ax1.plot(hours, frame["grid_kwh"] * 6, lw=1.25, color=colors["grid"], label="计划购电功率")
59     ax1.set_ylabel("功率 / kW")
60     ax1.grid(axis="y")
61     ax1.legend(ncol=3, loc="upper center", frameon=False)
62
63     discharge_kw = frame["discharge_kwh"] * 6
64     charge_kw = -frame["charge_kwh"] * 6
65     ax2.fill_between(hours, 0, discharge_kw, color=colors["discharge"], alpha=0.52, label="储能
66     放电")
67     ax2.fill_between(hours, 0, charge_kw, color=colors["charge"], alpha=0.52, label="储能充电")
68     ax2.set_ylabel("储能功率 / kW")

```

```

66     ax2.grid(axis="y")
67     ax2b = ax2.twinx()
68     ax2b.plot(hours, frame["price_yuan_per_kwh"], color=colors["price"], lw=1.0,
69               ls=":", label="分时电价")
70     ax2b.set_ylabel("电价 / (元 · kWh-${-1})")
71     handles, labels = ax2.get_legend_handles_labels()
72     handles2, labels2 = ax2b.get_legend_handles_labels()
73     ax2.legend(handles + handles2, labels + labels2, ncol=3, loc="upper center", frameon=False)
74
75     ax3.plot(hours, frame["soc_end_kwh"], color=colors["soc"], lw=1.7, label="储电量")
76     ax3.axhline(1200, color=colors["soc"], lw=0.7, ls="--", alpha=0.55, label="运行边界")
77     ax3.axhline(10800, color=colors["soc"], lw=0.7, ls="--", alpha=0.55)
78     ax3.fill_between(hours, 1200, 10800, color=colors["soc"], alpha=0.035)
79     ax3.set_ylabel("储电量 / kWh")
80     ax3.set_xlabel("时刻 / h")
81     ax3.set_xlim(0, 24)
82     ax3.set_xticks(np.arange(0, 25, 2))
83     ax3.set_ylim(0, 12000)
84     ax3.grid(axis="y")
85     ax3.legend(ncol=2, loc="upper center", frameon=False)
86     for ax in (ax1, ax2, ax2b, ax3):
87         ax.spines["top"].set_visible(False)
88     _save(fig, "q1_dispatch_and_soc")
89
90
91 def plot_model_growth() -> None:
92     fig, ax = plt.subplots(figsize=(7.35, 4.55))
93     ax.set_xlim(0, 10)
94     ax.set_ylim(0, 8)
95     ax.axis("off")
96     rows = [
97         (6.55, "统一物理骨架", "母线能量守恒 · SOC状态转移 · 容量/功率/互斥", "#D9EAF7", "#1
98         F4E78"),
99         (4.85, "问题一：信息完全", "144时段确定性MILP | LP连续松弛下界", "#E8F3E8", "#2E7D32")
100         ,
101         (3.15, "问题二：日前未知", "共享日前计划 + 情景紧急购电追索 | 跨日SOC", "#FFF1D6", "#
102         C77800"),
103         (1.45, "问题三、四：信息滚动", "光伏/价格因果更新 + 未执行时段MPC | 完全信息评价基准",
104         "#F3E8FA", "#7B2CBF"),
105     ]
106     for y, title, body, fill, edge in rows:
107         box = FancyBboxPatch((0.65, y), 8.7, 1.05, boxstyle="round,pad=0.03,rounding_size
108         =0.12",

```

```

104         linewidth=1.35, edgecolor=edge, facecolor=fill)
105     ax.add_patch(box)
106     ax.text(1.0, y + 0.7, title, fontsize=11, fontweight="bold", color=edge, va="center")
107     ax.text(1.0, y + 0.3, body, fontsize=9.3, color="#263238", va="center")
108     for y1, y2 in ((6.55, 5.9), (4.85, 4.2), (3.15, 2.5)):
109         ax.add_patch(FancyArrowPatch((5.0, y1), (5.0, y2), arrowstyle="-|>", mutation_scale=13,
110                                     linewidth=1.1, color="#607D8B"))
111     _save(fig, "model_growth_route")
112
113
114 def plot_q2() -> None:
115     z = np.load(ROOT / "results" / "q2_detail.npz", allow_pickle=True)
116     dates = pd.to_datetime(z["dates"])
117     start = int(z["official_start_index"][0])
118     frame = pd.DataFrame({
119         "date": dates[start:],
120         "计划购电费": z["planned_cost_yuan"][start:],
121         "紧急购电费": z["emergency_cost_yuan"][start:],
122         "紧急购电量": z["actual_emergency_kwh"][start:].sum(axis=1),
123     }).set_index("date")
124     monthly = frame.resample("ME").sum()
125     x = np.arange(len(monthly))
126     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7.35, 5.0), gridspec_kw={"hspace": 0.3})
127     ax1.bar(x, monthly["计划购电费"] / 1e4, color="#4C78A8", label="计划购电费")
128     ax1.bar(x, monthly["紧急购电费"] / 1e4, bottom=monthly["计划购电费"] / 1e4,
129            color="#E45756", label="紧急购电费")
130     ax1.set_ylabel("月费用 / 万元")
131     ax1.legend(ncol=2, frameon=False)
132     ax1.set_xticks(x, [f"{d.month}月" for d in monthly.index])
133     ax1.grid(axis="y")
134     ax2.bar(x, monthly["紧急购电量"] / 1e3, color="#F28E2B")
135     ax2.set_ylabel("紧急购电量 / MWh")
136     ax2.set_xticks(x, [f"{d.month}月" for d in monthly.index])
137     ax2.grid(axis="y")
138     for ax in (ax1, ax2):
139         ax.spines[["top", "right"]].set_visible(False)
140     _save(fig, "q2_monthly_cost_and_emergency")
141
142     chosen = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
143     fig, axes = plt.subplots(2, 2, figsize=(7.35, 5.2), sharex=True)
144     h = np.arange(144) / 6
145     for ax, day in zip(axes.flat, chosen):
146         i = int(np.flatnonzero(dates.strftime("%Y-%m-%d") == day)[0])

```

```

147     ax.plot(h, z["actual_net_kwh"][i] * 6, color="#243B53", lw=1.35, label="实际净负荷")
148     ax.plot(h, z["point_forecast_net_kwh"][i] * 6, color="#4C78A8", lw=1.1,
149             ls="--", label="日前预测")
150     ax.fill_between(h, 0, z["actual_emergency_kwh"][i] * 6,
151                    color="#E45756", alpha=0.42, label="紧急购电")
152     ax.text(0.02, 0.92, day, transform=ax.transAxes, fontsize=9, fontweight="bold")
153     ax.grid(axis="y")
154     ax.spines[["top", "right"]].set_visible(False)
155     axes[0, 0].set_ylabel("功率 / kW")
156     axes[1, 0].set_ylabel("功率 / kW")
157     axes[1, 0].set_xlabel("时刻 / h")
158     axes[1, 1].set_xlabel("时刻 / h")
159     handles, labels = axes[0, 0].get_legend_handles_labels()
160     fig.legend(handles, labels, ncol=3, frameon=False, loc="upper center", bbox_to_anchor=(0.5,
161         1.01))
162     fig.subplots_adjust(top=0.91)
163     _save(fig, "q2_representative_days")
164
165 def plot_q3() -> None:
166     report = json.loads((ROOT / "results" / "q3_results.json").read_text(encoding="utf-8"))
167     order = ["0_only", "0_6", "0_6_12", "0_6_12_18"]
168     labels = ["0时", "0/6时", "0/6/12时", "0/6/12/18时"]
169     ab = report["update_ablation"]
170     planned = np.array([ab[k]["planned_cost_yuan"] for k in order]) / 1e4
171     adjust = np.array([ab[k]["adjustment_cost_yuan"] for k in order]) / 1e4
172     emerg = np.array([ab[k]["emergency_cost_yuan"] for k in order]) / 1e4
173     energy = np.array([ab[k]["emergency_energy_kwh"] for k in order]) / 1e3
174     x = np.arange(4)
175     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.35, 3.55), gridspec_kw={"wspace": 0.3})
176     ax1.bar(x, planned, color="#4C78A8", label="初始计划费用")
177     ax1.bar(x, adjust, bottom=planned, color="#72B7B2", label="调整净费用")
178     ax1.bar(x, emerg, bottom=planned + adjust, color="#E45756", label="紧急购电费")
179     ax1.set_ylabel("全年费用 / 万元")
180     ax1.set_xticks(x, labels, rotation=16)
181     ax1.legend(frameon=False, fontsize=7.6, ncol=3, loc="upper center",
182               bbox_to_anchor=(0.5, 1.12))
183     ax1.grid(axis="y")
184     ax2.bar(x, energy, color=["#C9D6E4", "#94B6D2", "#5F95BE", "#2F6F9F"])
185     ax2.set_ylabel("紧急购电量 / MWh")
186     ax2.set_xticks(x, labels, rotation=16)
187     ax2.grid(axis="y")
188     for ax in (ax1, ax2):

```

```

189         ax.spines[["top", "right"]].set_visible(False)
190     _save(fig, "q3_update_ablation")
191
192     z = np.load(ROOT / "results" / "q3_detail.npz", allow_pickle=True)
193     dates = pd.to_datetime(z["dates"])
194     chosen = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
195     fig, axes = plt.subplots(2, 2, figsize=(7.35, 5.25), sharex=True)
196     h = np.arange(144) / 6
197     for ax, day in zip(axes.flat, chosen):
198         i = int(np.flatnonzero(dates.strftime("%Y-%m-%d") == day)[0])
199         net = z["effective_grid"][i] + z["emergency"][i] + z["discharge"][i] \
200             - z["charge"][i] - z["curtailment"][i]
201         ax.plot(h, net * 6, color="#243B53", lw=1.25, label="实际净负荷")
202         for oi, origin in enumerate((0, 36, 72, 108)):
203             vals = z["load_forecasts"][i, oi] - z["pv_forecasts"][i, oi]
204             ax.plot(h[origin:], vals[origin:] * 6, lw=0.9, alpha=0.74,
205                 label=f"{origin//6}时预测")
206             ax.fill_between(h, 0, z["emergency"][i] * 6, color="#E45756", alpha=0.34,
207                 label="紧急购电")
208             ax.text(0.02, 0.92, day, transform=ax.transAxes, fontsize=9, fontweight="bold")
209             ax.grid(axis="y")
210             ax.spines[["top", "right"]].set_visible(False)
211     axes[0, 0].set_ylabel("功率 / kW")
212     axes[1, 0].set_ylabel("功率 / kW")
213     axes[1, 0].set_xlabel("时刻 / h")
214     axes[1, 1].set_xlabel("时刻 / h")
215     handles, labels2 = axes[0, 0].get_legend_handles_labels()
216     fig.legend(handles, labels2, ncol=3, frameon=False, loc="upper center", bbox_to_anchor
217         =(0.5, 1.01))
218     fig.subplots_adjust(top=0.88)
219     _save(fig, "q3_forecast_updates")
220
221 def plot_q4() -> None:
222     report = json.loads((ROOT / "results" / "q4_results.json").read_text(encoding="utf-8"))
223     k = report["key_results"]
224     strategies = ["因果在线", "完全价格信息", "全信息下界"]
225     q42 = [k["q4_2_causal"]["total_cost_yuan"],
226         k["q4_2_perfect_price"]["total_cost_yuan"],
227         k["full_information_oracle_lower_bound"]["total_cost_yuan"]]
228     q43 = [k["q4_3_causal"]["total_cost_yuan"],
229         k["q4_3_perfect_price"]["total_cost_yuan"],
230         k["full_information_oracle_lower_bound"]["total_cost_yuan"]]

```

```

231     x = np.arange(3); width = 0.34
232     fig, ax = plt.subplots(figsize=(7.35, 3.8))
233     b1 = ax.bar(x - width/2, np.array(q42)/1e4, width, color="#4C78A8", label="问题4-2")
234     b2 = ax.bar(x + width/2, np.array(q43)/1e4, width, color="#F28E2B", label="问题4-3")
235     ax.set_xticks(x, strategies)
236     ax.set_ylabel("正式期总费用 / 万元")
237     ax.grid(axis="y")
238     ax.spines[["top", "right"]].set_visible(False)
239     ax.legend(frameon=False, ncol=2, loc="upper right")
240     for bars in (b1, b2):
241         for bar in bars:
242             ax.text(bar.get_x()+bar.get_width()/2, bar.get_height(), f"{bar.get_height():.1f}",
243                     ha="center", va="bottom", fontsize=8)
244     _save(fig, "q4_information_value")
245
246
247 def main() -> None:
248     setup_style()
249     plot_model_growth()
250     if (ROOT / "results" / "q1_detail.csv").exists():
251         plot_q1()
252     if (ROOT / "results" / "q2_detail.npz").exists():
253         plot_q2()
254     if (ROOT / "results" / "q3_detail.npz").exists():
255         plot_q3()
256     if (ROOT / "results" / "q4_detail.npz").exists():
257         q4_dates = np.load(ROOT / "results" / "q4_detail.npz", allow_pickle=True)["dates"]
258         if len(q4_dates) == 365:
259             plot_q4()
260
261
262 if __name__ == "__main__":
263     main()

```